The Mothership has Landed

Adding <=> to the Library

Document #:	P1614R1
Date:	2019-06-16
Project:	Programming Language C++ LWG
Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

[P1614R0] took the route of adding the new comparison operators as hidden friends. This paper instead preserves the current method of declaring comparisons - typically as non-member functions. See friendship for a more thorough discussion.

The comparisons between `unique_ptr<T, D>` and `nullptr` were originally removed and replaced with a `<=>`, but this was reverted.

Additionally, R0 used the `3WAY<R>` wording from [P1186R1], which was removed in the subsequent [P1186R2] - so the relevant wording for the fallback objects was changed as well.

§ 2 Introduction

The work of integrating `operator<=>` into the library has been performed by multiple different papers, each addressing a different aspect of the integration. In the interest of streamlining review by the Library Working Group, the wording has been combined into a single paper. This is that paper.

In San Diego and Kona, several papers were approved by LEWG adding functionality to the library related to comparisons. What follows is the list of those papers, in alphabetical order, with a brief description of what those papers are. The complete motivation and design rationale for each can be found within the papers themselves.

— [P0790R2] - adding `operator<=>` to the standard library types whose behavior is not dependent on a template parameter.

— [P0891R2] - making the `XXX_order` algorithms customization points and introducing `compare_XXX_order_fallback` algorithms that preferentially invoke the former algorithm and fallback to synthesizing an ordering from `==` and `<` (using the rules from

[P1186R1]).

— [P1154R1] - adding the type trait `has_strong_structural_equality<T>` (useful to check if a type can be used as a non-type template parameter).

— [P1188R0] - adding the type trait `compare_three_way_result<T>`, the concepts `ThreeWayComparable<T>` and `ThreeWayComparableWith<T,U>`, removing the algorithm `compare_3way` and replacing it with a function comparison object `compare_three_way` (i.e. the `<=>` version of `std::ranges::less`).

— [P1189R0] - adding `operator<=>` to the standard library types whose behavior is dependent on a template parameter, removing those equality operators made redundant by [P1185R2] and defaulting `operator==` where appropriate.

— [P1191R0] - adding equality to several previously incomparable standard library types.

— [P1295R0] - adding equality and `common_type` for the comparison categories.

— [P1380R1] - extending the floating point customization points for `strong_order` and `weak_order`.

3 Friendship

LEWG's unanimous preference was that the new `operator<=>`s be declared as hidden friends. It would follow therefore that we would move the `operator==`s to be declared the same way as well, since it would be pretty odd if the two different comparison operators had different semantics.

However, a few issues have come up with this approach that are worth presenting clearly here.

3.1 Was well-formed, now ill-formed

Here is an example that came up while I attempted to implement these changes to measure any improvements in build time that might come up. This is a reproduction from the LLVM codebase:

```
struct StringRef {
    StringRef(std::string const&); // NB: non-explicit
    operator std::string() const;  // NB: non-explicit
};
bool operator==(StringRef, StringRef);

bool f(StringRef a, std::string b) {
    return a == b; // (*)
}
```

In C++17, the marked line is well-formed. The `operator==` for `basic_string` is a non-member function template, and so would not be considered a candidate; the only viable candidate is the `operator==` taking two `StringRef`s. With the proposed changes, the `operator==` for `basic_string` becomes a non-member hidden friend, *non-template*, which makes it a candidate (converting `a` to a `string`). That candidate is ambiguous with the `operator==(StringRef, StringRef)` candidate - each requires a conversion in one argument, so the call becomes ill-formed.

Many people might consider such a type - implicitly convertible in both directions (note that `string` to `string_view` is implicit, but `string_view` to `string` is explicit) - questionable. But this is still a breaking change to consider.

3.2 Was ill-formed, now well-formed

```
bool ref_equal(std::reference_wrapper<std::string> a,
               std::reference_wrapper<std::string> b)
{
    return a == b;
}
```

The comparisons for `std::reference_wrapper<T>` are very strange. It's not that this type is comparable based on *whether* `T` is comparable. It's actually that this type is comparable based on *how* `T` is comparable. We can compare `std::reference_wrapper<int>`s, but we cannot compare `std::reference_wrapper<std::string>`s because the comparisons for `basic_string` are non-member function templates. That's just weird. This change wouldn't actually resolve that weirdness generally (it wouldn't affect any user types whose comparisons are non-member function templates), but it would at least reduce it for the standard library. Arguably an improvement.

However, the more interesting case is:

```
bool is42(std::variant<int, std::string> const& v) {
    return v == 42; // (*)
}
```

In C++17, the `operator==` for `variant` is a non-member function template and is thus not a viable candidate for the marked line. That check is ill-formed. With the proposed changes, the `operator==` for `variant` becomes a non-member hidden friend, *non-template*, which makes it a candidate (converting `42` to a `variant<int, string>`). Many would argue that

this a fix, since both `variant<int, string> v = 42;` and `v = 42;` are already well-formed, so it is surely reasonable that `v == 42` is as well.

But we already had a proposal to do precisely this: [P1201R0], which failed to gain consensus in LEWGI in San Diego (vote was 2-6-2-3-3).

3.3 Alternatives

The benefit of the hidden friend technique wasn't the only way to achieve the ultimate goal of reducing the overload candidate set. Casey Carter suggested another:

```
template<class T, class Traits = char_traits<T>, class Alloc = allocator<T>> class basic_string;

namespace __foo {
  struct __tag {};

  template<class T, class Traits, class Alloc>
  bool operator==(const basic_string<T, Traits, Alloc>&, const basic_string<T, Traits, Alloc>&);

  /* ... */
}

template<class T, class Traits, class Alloc> class basic_string : private __foo::__tag {
  /* ... */
};
```

That is, we keep `basic_string`'s comparisons as non-member function templates – but we move them into a different namespace that is *only* associated with `basic_string`. This is an interesting direction to take, but is novel and has some specification burden.

3.4 Proposed Direction

Ultimately, the goal here is to add `<=>` to all the types in the standard library. While I think the goal of reducing the candidate set for comparisons with standard library types is absolutely worth pursuing, it is a completely orthogonal goal and can be addressed by a different proposal in the future.

Given that we've said that users aren't allowed to take the address of most standard library functions, Casey's proposed implementation might even be valid under today's wording for those standard library class templates whose comparisons are non-member templates, so I'd encourage implementors to experiment there.

The direction this paper is taking is the path of least resistance: keep all the comparison operators as they are. Add `<=>` in the same form that `<` appears today. With a few exceptions: those types for which adding a defaulted `operator==` would allow them to be used as non-type template parameters after [P0732R2] (and only those types) will have their comparisons implemented as hidden friends.

4 Acknowledgements

Thank you to all the paper authors that have committed time to making sure all this works: Gašper Ažman, Walter Brown, Lawrence Crowl, Tomasz Kamiński, Arthur O'Dwyer, Jeff Snyder, David Stone, and Herb Sutter.

Thank you to Casey Carter for the tremendous wording review.

5 Wording

5.1 Clause 16: Library Introduction

Change 16.4.2.1/2 [expos.only.func]:

> The following ~~function is~~ are defined for exposition only to aid in the specification of the library:

and append:

```
constexpr auto synth-3way =
  []<class T, class U>(const T& t, const U& u)
    requires requires {
      { t < u } -> bool;
      { u < t } -> bool;
    }
  {
    if constexpr (ThreeWayComparableWith<T, U>) {
      return t <=> u;
    } else {
      if (t < u) return weak_ordering::less;
      if (u < t) return weak_ordering::greater;
      return weak_ordering::equivalent;
    }
  };
```

```
template<class T, class U=T>
using synth-3way-result = decltype(synth-3way(declval<T&>(), declval<U&>()));
```

Remove all of 16.4.2.3 [operators], which begins:

~~In this library, whenever a declaration is provided for an `operator !=`, `operator <=`, or `operator >=` for a type `T`, its requirements and semantics are as follows, unless explicitly specified otherwise.~~

§ 5.2 Clause 17: Language support library

Added: `compare_three_way_result`, concepts `ThreeWayComparable` and `ThreeWayComparableWith`, `compare_three_way` and `compare_XXX_order_fallback`

Changed operators for: `type_info`

Respecified: `strong_order()`, `weak_order()`, and `partial_order()`

Removed: `compare_3way()`, `strong_equal()`, and `weak_equal()`

In 17.7.2 [type.info], remove `operator!=`:

```
namespace std {
  class type_info {
  public:
    virtual ~type_info();
    bool operator==(const type_info& rhs) const noexcept;
-   bool operator!=(const type_info& rhs) const noexcept;
    bool before(const type_info& rhs) const noexcept;
    size_t hash_code() const noexcept;
    const char* name() const noexcept;
    type_info(const type_info& rhs) = delete; // cannot be copied
    type_info& operator=(const type_info& rhs) = delete; // cannot be copied
  };
}
```

and:

```
bool operator==(const type_info& rhs) const noexcept;
```

2 *Effects*: Compares the current object with `rhs`.

3 *Returns*: `true` if the two values describe the same type.

~~`bool operator!=(const type_info& rhs) const noexcept;`~~

4 ~~*Returns:* `!(*this == rhs)`.~~

Add into 17.11.1 [compare.syn]:

```
namespace std {
  // [cmp.categories], comparison category types
  class weak_equality;
  class strong_equality;
  class partial_ordering;
  class weak_ordering;
  class strong_ordering;

  // named comparison functions
  constexpr bool is_eq  (weak_equality cmp) noexcept    { return cmp == 0; }
  constexpr bool is_neq (weak_equality cmp) noexcept    { return cmp != 0; }
  constexpr bool is_lt  (partial_ordering cmp) noexcept { return cmp < 0; }
  constexpr bool is_lteq(partial_ordering cmp) noexcept { return cmp <= 0; }
  constexpr bool is_gt  (partial_ordering cmp) noexcept { return cmp > 0; }
  constexpr bool is_gteq(partial_ordering cmp) noexcept { return cmp >= 0; }

  // [cmp.common], common comparison category type
  template<class... Ts>
  struct common_comparison_category {
    using type = see below;
  };
  template<class... Ts>
    using common_comparison_category_t = typename common_comparison_category<Ts...>::type;

+ // [cmp.concept], concept ThreeWayComparable
+ template<class T, class Cat = partial_ordering>
+   concept ThreeWayComparable = see below;
+ template<class T, class U, class Cat = partial_ordering>
+   concept ThreeWayComparableWith = see below;
```

```
+
+ // [cmp.result], spaceship invocation result
+ template<class T, class U = T> struct compare_three_way_result;
+
+ template<class T, class U = T>
+   using compare_three_way_result_t = typename compare_three_way_result<T, U>::type;
+
+ // [cmp.object], spaceship object
+ struct compare_three_way;

  // [cmp.alg], comparison algorithms
- template<class T> constexpr strong_ordering strong_order(const T& a, const T& b);
- template<class T> constexpr weak_ordering weak_order(const T& a, const T& b);
- template<class T> constexpr partial_ordering partial_order(const T& a, const T& b);
- template<class T> constexpr strong_equality strong_equal(const T& a, const T& b);
- template<class T> constexpr weak_equality weak_equal(const T& a, const T& b);
+ inline namespace unspecified {
+   inline constexpr unspecified strong_order = unspecified;
+   inline constexpr unspecified weak_order = unspecified;
+   inline constexpr unspecified partial_order = unspecified;
+   inline constexpr unspecified compare_strong_order_fallback = unspecified;
+   inline constexpr unspecified compare_weak_order_fallback = unspecified;
+   inline constexpr unspecified compare_partial_order_fallback = unspecified;
+ }
}
```

Change 17.11.2.2 [cmp.weakeq]:

```
namespace std {
  class weak_equality {
    int value;  // exposition only

    [...]

    // comparisons
    friend constexpr bool operator==(weak_equality v, unspecified) noexcept;
-   friend constexpr bool operator!=(weak_equality v, unspecified) noexcept;
-   friend constexpr bool operator==(unspecified, weak_equality v) noexcept;
-   friend constexpr bool operator!=(unspecified, weak_equality v) noexcept;
+   friend constexpr bool operator==(weak_equality v, weak_equality w) noexcept = default;
    friend constexpr weak_equality operator<=>(weak_equality v, unspecified) noexcept;
    friend constexpr weak_equality operator<=>(unspecified, weak_equality v) noexcept;
  };

  [...]
}
```

and remove those functions from the description:

```
constexpr bool operator==(weak_equality v, unspecified) noexcept;
~~constexpr bool operator==(unspecified, weak_equality v) noexcept;~~
```

2 *Returns*: `v.value == 0`.

```
~~constexpr bool operator!=(weak_equality v, unspecified) noexcept;~~
~~constexpr bool operator!=(unspecified, weak_equality v) noexcept;~~
```

3 ~~*Returns:* `v.value != 0`.~~

Change 17.11.2.3 [cmp.strongeq]:

```
namespace std {
  class strong_equality {
    int value;  // exposition only

    [...]

    // comparisons
    friend constexpr bool operator==(strong_equality v, unspecified) noexcept;
-   friend constexpr bool operator!=(strong_equality v, unspecified) noexcept;
-   friend constexpr bool operator==(unspecified, strong_equality v) noexcept;
-   friend constexpr bool operator!=(unspecified, strong_equality v) noexcept;
+   friend constexpr bool operator==(strong_equality v, strong_equality w) noexcept = default;
    friend constexpr strong_equality operator<=>(strong_equality v, unspecified) noexcept;
    friend constexpr strong_equality operator<=>(unspecified, strong_equality v) noexcept;
  };

  [...]
}
```

and remove those functions from the description:

```
constexpr bool operator==(strong_equality v, unspecified) noexcept;
constexpr bool operator==(unspecified, strong_equality v) noexcept;
```

3 *Returns*: `v.value == 0`.

```
constexpr bool operator!=(strong_equality v, unspecified) noexcept;
constexpr bool operator!=(unspecified, strong_equality v) noexcept;
```

4 *Returns*: `v.value != 0`.

Change 17.11.2.4 [cmp.partialord]:

```
namespace std {
  class partial_ordering {
    int value;          // exposition only
    bool is_ordered;    // exposition only

    [...]

    // conversion
    constexpr operator weak_equality() const noexcept;

    // comparisons
    friend constexpr bool operator==(partial_ordering v, unspecified) noexcept;
-   friend constexpr bool operator!=(partial_ordering v, unspecified) noexcept;
+   friend constexpr bool operator==(partial_ordering v, partial_ordering w) noexcept = default;
    friend constexpr bool operator< (partial_ordering v, unspecified) noexcept;
    friend constexpr bool operator> (partial_ordering v, unspecified) noexcept;
    friend constexpr bool operator<=(partial_ordering v, unspecified) noexcept;
    friend constexpr bool operator>=(partial_ordering v, unspecified) noexcept;
-   friend constexpr bool operator==(unspecified, partial_ordering v) noexcept;
-   friend constexpr bool operator!=(unspecified, partial_ordering v) noexcept;
    friend constexpr bool operator< (unspecified, partial_ordering v) noexcept;
    friend constexpr bool operator> (unspecified, partial_ordering v) noexcept;
    friend constexpr bool operator<=(unspecified, partial_ordering v) noexcept;
    friend constexpr bool operator>=(unspecified, partial_ordering v) noexcept;
    friend constexpr partial_ordering operator<=>(partial_ordering v, unspecified) noexcept;
    friend constexpr partial_ordering operator<=>(unspecified, partial_ordering v) noexcept;
  };

  [...]
}
```

Remove just the extra `==` and `!=` operators in 17.11.2.4 [cmp.partialord]/4-5:

```
constexpr bool operator==(partial_ordering v, unspecified) noexcept;
constexpr bool operator< (partial_ordering v, unspecified) noexcept;
constexpr bool operator> (partial_ordering v, unspecified) noexcept;
constexpr bool operator<=(partial_ordering v, unspecified) noexcept;
constexpr bool operator>=(partial_ordering v, unspecified) noexcept;
```

3 *Returns*: For `operator@, v.is_ordered && v.value @ 0`.

```
constexpr bool operator==(unspecified, partial_ordering v) noexcept;
constexpr bool operator< (unspecified, partial_ordering v) noexcept;
constexpr bool operator> (unspecified, partial_ordering v) noexcept;
constexpr bool operator<=(unspecified, partial_ordering v) noexcept;
constexpr bool operator>=(unspecified, partial_ordering v) noexcept;
```

4 *Returns*: For `operator@, v.is_ordered && 0 @ v.value`.

```
constexpr bool operator!=(partial_ordering v, unspecified) noexcept;
constexpr bool operator!=(unspecified, partial_ordering v) noexcept;
```

5 *Returns*: For `operator@, !v.is_ordered || v.value != 0`.

Change 17.11.2.5 [cmp.weakord]:

```
namespace std {
  class weak_ordering {
    int value;  // exposition only

    [...]
    // comparisons
    friend constexpr bool operator==(weak_ordering v, unspecified) noexcept;
+   friend constexpr bool operator==(weak_ordering v, weak_ordering w) noexcept = default;
-   friend constexpr bool operator!=(weak_ordering v, unspecified) noexcept;
    friend constexpr bool operator< (weak_ordering v, unspecified) noexcept;
    friend constexpr bool operator> (weak_ordering v, unspecified) noexcept;
    friend constexpr bool operator<=(weak_ordering v, unspecified) noexcept;
    friend constexpr bool operator>=(weak_ordering v, unspecified) noexcept;
```

```
-   friend constexpr bool operator==(unspecified, weak_ordering v) noexcept;
-   friend constexpr bool operator!=(unspecified, weak_ordering v) noexcept;
    friend constexpr bool operator< (unspecified, weak_ordering v) noexcept;
    friend constexpr bool operator> (unspecified, weak_ordering v) noexcept;
    friend constexpr bool operator<=(unspecified, weak_ordering v) noexcept;
    friend constexpr bool operator>=(unspecified, weak_ordering v) noexcept;
    friend constexpr weak_ordering operator<=>(weak_ordering v, unspecified) noexcept;
    friend constexpr weak_ordering operator<=>(unspecified, weak_ordering v) noexcept;
  };

  [...]
}
```

Remove just the extra `==` and `!=` operators from 17.11.2.5 [cmp.weakord]/4 and /5:

```
constexpr bool operator==(weak_ordering v, unspecified) noexcept;
constexpr bool operator!=(weak_ordering v, unspecified) noexcept;
constexpr bool operator< (weak_ordering v, unspecified) noexcept;
constexpr bool operator> (weak_ordering v, unspecified) noexcept;
constexpr bool operator<=(weak_ordering v, unspecified) noexcept;
constexpr bool operator>=(weak_ordering v, unspecified) noexcept;
```

4 *Returns*: `v.value @ 0` for `operator@`.

```
constexpr bool operator==(unspecified, weak_ordering v) noexcept;
constexpr bool operator!=(unspecified, weak_ordering v) noexcept;
constexpr bool operator< (unspecified, weak_ordering v) noexcept;
constexpr bool operator> (unspecified, weak_ordering v) noexcept;
constexpr bool operator<=(unspecified, weak_ordering v) noexcept;
constexpr bool operator>=(unspecified, weak_ordering v) noexcept;
```

5 *Returns*: `0 @ v.value` for `operator@`.

Change 17.11.2.6 [cmp.strongord]:

```
namespace std {
  class strong_ordering {
    int value;  // exposition only

    [...]

    // comparisons
    friend constexpr bool operator==(strong_ordering v, unspecified) noexcept;
+   friend constexpr bool operator==(strong_ordering v, strong_ordering w) noexcept = default;
-   friend constexpr bool operator!=(strong_ordering v, unspecified) noexcept;
    friend constexpr bool operator< (strong_ordering v, unspecified) noexcept;
    friend constexpr bool operator> (strong_ordering v, unspecified) noexcept;
    friend constexpr bool operator<=(strong_ordering v, unspecified) noexcept;
    friend constexpr bool operator>=(strong_ordering v, unspecified) noexcept;
-   friend constexpr bool operator==(unspecified, strong_ordering v) noexcept;
-   friend constexpr bool operator!=(unspecified, strong_ordering v) noexcept;
    friend constexpr bool operator< (unspecified, strong_ordering v) noexcept;
    friend constexpr bool operator> (unspecified, strong_ordering v) noexcept;
    friend constexpr bool operator<=(unspecified, strong_ordering v) noexcept;
    friend constexpr bool operator>=(unspecified, strong_ordering v) noexcept;
    friend constexpr strong_ordering operator<=>(strong_ordering v, unspecified) noexcept;
    friend constexpr strong_ordering operator<=>(unspecified, strong_ordering v) noexcept;
  };

  [...]
}
```

Remove just the extra `==` and `!=` operators from 17.11.2.6 [cmp.strongord]/6 and /7:

```
constexpr bool operator==(strong_ordering v, unspecified) noexcept;
constexpr bool operator!=(strong_ordering v, unspecified) noexcept;
constexpr bool operator< (strong_ordering v, unspecified) noexcept;
constexpr bool operator> (strong_ordering v, unspecified) noexcept;
constexpr bool operator<=(strong_ordering v, unspecified) noexcept;
constexpr bool operator>=(strong_ordering v, unspecified) noexcept;
```

6 *Returns*: `v.value @ 0` for `operator@`.

```
constexpr bool operator==(unspecified, strong_ordering v) noexcept;
constexpr bool operator!=(unspecified, strong_ordering v) noexcept;
constexpr bool operator< (unspecified, strong_ordering v) noexcept;
constexpr bool operator> (unspecified, strong_ordering v) noexcept;
constexpr bool operator<=(unspecified, strong_ordering v) noexcept;
constexpr bool operator>=(unspecified, strong_ordering v) noexcept;
```

7 *Returns*: `0 @ v.value` for `operator@`.

Add a new subclause [cmp.concept] “concept ThreeWayComparable”:

```
template <typename T, typename Cat>
  concept compares-as = // exposition only
    Same<common_comparison_category_t<T, Cat>, Cat>;

template<class T, class U>
  concept partially-ordered-with = // exposition only
    requires(const remove_reference_t<T>& t,
             const remove_reference_t<U>& u) {
      { t < u } -> Boolean;
      { t > u } -> Boolean;
      { t <= u } -> Boolean;
      { t >= u } -> Boolean;
      { u < t } -> Boolean;
      { u > t } -> Boolean;
      { u <= t } -> Boolean;
      { u >= t } -> Boolean;
    };
```

1 Let `t` and `u` be lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>` respectively. *`partially-ordered-with<T, U>`* is satisfied only if:

(1.1) — `t < u`, `t <= u`, `t > u`, `t >= u`, `u < t`, `u <= t`, `u > t`, and `u >= t` have the same domain.

(1.2) — `bool(t < u) == bool(u > t)`

(1.3) — `bool(u < t) == bool(t > u)`

(1.4) — `bool(t <= u) == bool(u >= t)`

(1.5) — `bool(u <= t) == bool(t >= u)`

```
template <typename T, typename Cat = partial_ordering>
  concept ThreeWayComparable =
    weakly-equality-comparable-with<T, T> &&
    (!ConvertibleTo<Cat, partial_ordering> || partially-ordered-with<T, T>) &&
    requires(const remove_reference_t<T>& a,
             const remove_reference_t<T>& b) {
      { a <=> b } -> compares-as<Cat>;
    };
```

2 Let `a` and `b` be lvalues of type `const remove_reference_t<T>`. `T` and `Cat` model `ThreeWayComparable<T, Cat>` only if:

(2.1) — `(a <=> b) == 0) == bool(a == b)`.

(2.2) — `(a <=> b) != 0) == bool(a != b)`.

(2.3) — `((a <=> b) <=> 0)` and `(0 <=> (b <=> a))` are equal

(2.4) — If `Cat` is convertible to `strong_equality`, `T` models `EqualityComparable` ([concept.equalitycomparable]).

(2.5) — If `Cat` is convertible to `partial_ordering`:

(2.5.1) — `(a <=> b) < 0) == bool(a < b)`.

(2.5.2) — `(a <=> b) > 0) == bool(a > b)`.

(2.5.3) — `(a <=> b) <= 0) == bool(a <= b)`.

(2.5.4) — `(a <=> b) >= 0) == bool(a >= b)`.

(2.5.5) — If `Cat` is convertible to `strong_ordering`, `T` models `StrictTotallyOrdered` ([concept.stricttotallyordered]).

```
template <typename T, typename U,
          typename Cat = partial_ordering>
  concept ThreeWayComparableWith =
    weakly-equality-comparable-with<T, U> &&
    (!ConvertibleTo<Cat, partial_ordering> || partially-ordered-with<T, U>) &&
    ThreeWayComparable<T, Cat> &&
    ThreeWayComparable<U, Cat> &&
    CommonReference<const remove_reference_t<T>&, const remove_reference_t<U>&> &&
    ThreeWayComparable<
      common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>,
```

```
    Cat> &&
  requires(const remove_reference_t<T>& t,
           const remove_reference_t<U>& u) {
    { t <=> u } -> compares-as<Cat>;
    { u <=> t } -> compares-as<Cat>;
  };
```

3 Let `t` and `u` be lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>`, respectively. Let `C` be `common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>`. `T`, `U`, and `Cat` model `ThreeWayComparableWith<T, U, Cat>` only if:

(3.1) — `t <=> u` and `u <=> t` have the same domain.

(3.2) — `((t <=> u) <=> 0)` and `(0 <=> (u <=> t))` are equal

(3.3) — `(t <=> u == 0) == bool(t == u)`.

(3.4) — `(t <=> u != 0) == bool(t != u)`.

(3.5) — `Cat(t <=> u) == Cat(C(t) <=> C(u))`.

(3.6) — If `Cat` is convertible to `strong_equality`, `T` and `U` model `EqualityComparableWith<T, U>` ([concepts.equalitycomparable]).

(3.7) — If `Cat` is convertible to `partial_ordering`:

(3.7.1) — `(t <=> u < 0) == bool(t < u)`

(3.7.2) — `(t <=> u > 0) == bool(t > u)`

(3.7.3) — `(t <=> u <= 0) == bool(t <= u)`

(3.7.4) — `(t <=> u >= 0) == bool(t >= u)`

(3.8) — If `Cat` is convertible to `strong_ordering`, `T` and `U` model `StrictTotallyOrderedWith<T, U>` ([concepts.stricttotallyordered]).

Add a new subclause [cmp.result] “spaceship invocation result”:

The behavior of a program that adds specializations for the `compare_three_way_result` template defined in this subclause is undefined.

For the `compare_three_way_result` type trait applied to the types `T` and `U`, let `t` and `u` denote lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>`, respectively. If the expression `t <=> u` is well-formed when treated as an unevaluted operand ([expr.context]), the member *typedef-name* `type` denotes the type `decltype(t <=> u)`. Otherwise, there is no member `type`.

Add a new subclause [cmp.object] “spaceship object”:

1 In this subclause, `BUILTIN_PTR_3WAY(T, U)` for types `T` and `U` is a boolean constant expression. `BUILTIN_PTR_3WAY(T, U)` is `true` if and only if `<=>` in the expression `declval<T>() <=> declval<U>()` resolves to a built-in operator comparing pointers.

```
struct compare_three_way {
  template<class T, class U>
    requires ThreeWayComparableWith<T,U> || BUILTIN_PTR_3WAY(T, U)
  constexpr auto operator()(T&& t, U&& u) const;

  using is_transparent = unspecified;
};
```

2 *Expects*: If the expression `std::forward<T>(t) <=> std::forward<U>(u)` results in a call to a built-in operator `<=>` comparing pointers of type `P`, the conversion sequences from both `T` and `U` to `P` are equality-preserving ([concepts.equality]).

3 *Effects*:

(3.1) — If the expression `std::forward<T>(t) <=> std::forward<U>(u)` results in a call to a built-in operator `<=>` comparing pointers of type `P`: returns `strong_ordering::less` if (the converted value of) `t` precedes `u` in the implementation-

defined strict total order ([range.cmp]) over pointers of type `P`, `strong_ordering::greater` if `u` precedes `t`, and otherwise `strong_ordering::equal`.

(3.2) — Otherwise, equivalent to: `return std::forward<T>(t) <=> std::forward<U>(u);`

4 In addition to being available via inclusion of the `<compare>` header, the class `compare_three_way` is available when the header `<functional>` is included.

Replace the entirety of 17.11.4 [cmp.alg]. This section had the original design for `strong_order()`, `weak_order()`, `partial_order()`, `strong_equal()`, and `weak_equal()`. The new wording makes them CPOs.

1 The name `strong_order` denotes a customization point object ([customization.point.object]). The expression `strong_order(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to the following:

(1.1) — If the decayed types of `E` and `F` differ, `strong_order(E, F)` is ill-formed.

(1.2) — Otherwise, `strong_ordering(strong_order(E, F))` if it is a well-formed expression with overload resolution performed in a context that does not include a declaration of `std::strong_order`.

(1.3) — Otherwise, if the decayed type `T` of `E` and `F` is a floating point type, yields a value of type `strong_ordering` that is consistent with the ordering observed by `T`'s comparison operators, and if `numeric_limits<T>::is_iec559` is `true` is additionally consistent with the totalOrder operation as specified in ISO/IEC/IEEE 60599.

(1.4) — Otherwise, `strong_ordering(E <=> F)` if it is a well-formed expression.

(1.5) — Otherwise, `strong_order(E, F)` is ill-formed. [*Note*: This case can result in substitution failure when `strong_order(E, F)` appears in the immediate context of a template instantiation. —*end note*]

2 The name `weak_order` denotes a customization point object ([customization.point.object]). The expression `weak_order(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to the following:

(2.1) — If the decayed types of `E` and `F` differ, `weak_order(E, F)` is ill-formed.

(2.2) — Otherwise, `weak_ordering(weak_order(E, F))` if it is a well-formed expression with overload resolution performed in a context that does not include a declaration of `std::weak_order`.

(2.3) — Otherwise, if the decayed type `T` of `E` and `F` is a floating point type, yields a value of type `weak_ordering` that is consistent with the ordering observed by `T`'s comparison operators and `strong_order`, and if `numeric_liits<T>::is_iec559` is `true` is additionally consistent with the following equivalence classes, ordered from lesser to greater:

(2.3.1) — Together, all negative NaN values

(2.3.2) — Negative infinity

(2.3.3) — Each normal negative value

(2.3.4) — Each subnormal negative value

(2.3.5) — Together, both zero values

(2.3.6) — Each subnormal positive value

(2.3.7) — Each normal positive value

(2.3.8) — Positive infinity

(2.3.9) — Together, all positive NaN values

(2.4) — Otherwise, `weak_ordering(E <=> F)` if it is a well-formed expression.

(2.5) — Otherwise, `weak_ordering(strong_order(E, F))` if it is a well-formed expression.

(2.6) — Otherwise, `weak_order(E, F)` is ill-formed. [*Note*: This case can result in substitution failure when `std::weak_order(E, F)` appears in the immediate context of a template instantiation. —*end note*]

3 The name `partial_order` denotes a customization point object ([customization.point.object]). The expression `partial_order(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to the following:

(3.1) — If the decayed types of `E` and `F` differ, `partial_order(E, F)` is ill-formed.

(3.2) — Otherwise, `partial_ordering(partial_order(E, F))` if it is a well-formed expression with overload resolution performed in a context that does not include a declaration of `std::partial_order`.

(3.3) — Otherwise, `partial_ordering(E <=> F)` if it is a well-formed expression.

(3.4) — Otherwise, `partial_ordering(weak_order(E, F))` if it is a well-formed expression.

(3.5) — Otherwise, `partial_order(E, F)` is ill-formed. [*Note*: This case can result in substitution failure when `std::partial_order(E, F)` appears in the immediate context of a template instantiation. —*end note*]

4 The name `compare_strong_order_fallback` denotes a comparison customization point ([customization.point.object]) object. The expression `compare_strong_order_fallback(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to:

(4.1) — If the decayed types of `E` and `F` differ, `compare_strong_order_fallback(E, F)` is ill-formed.

(4.2) — Otherwise, `strong_order(E, F)` if it is a well-formed expression.

(4.3) — Otherwise, if the expressions `E == F` and `E < F` are each well-formed and convertible to bool,

```
E == F ? strong_ordering::equal :
E < F  ? strong_ordering::less :
         strong_ordering::greater
```

except that `E` and `F` are only evaluated once.

(4.4) — Otherwise, `compare_strong_order_fallback(E, F)` is ill-formed.

5 The name `compare_weak_order_fallback` denotes a customization point object ([customization.point.object]). The expression `compare_weak_order_fallback(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to:

(5.1) — If the decayed types of `E` and `F` differ, `compare_weak_order_fallback(E, F)` is ill-formed.

(5.2) — Otherwise, `weak_order(E, F)` if it is a well-formed expression.

(5.3) — Otherwise, if the expressions `E == F` and `E < F` are each well-formed and convertible to bool,

```
E == F ? weak_ordering::equal :
E < F  ? weak_ordering::less :
         weak_ordering::greater
```

except that `E` and `F` are only evaluated once.

(5.4) — Otherwise, `compare_weak_order_fallback(E, F)` is ill-formed.

6 The name `compare_partial_order_fallback` denotes a customization point object ([customization.point.object]). The expression `compare_partial_order_fallback(E, F)` for some subexpressions `E` and `F` is expression-equivalent ([defns.expression-equivalent]) to:

(6.1) — If the decayed types of `E` and `F` differ, `compare_partial_order_fallback(E, F)` is ill-formed.

(6.2) — Otherwise, `partial_order(E, F)` if it is a well-formed expression.

(6.3) — Otherwise, if the expressions `E == F` and `E < F` are each well-formed and convertible to bool,

```
E == F ? partial_ordering::equivalent :
E < F  ? partial_ordering::less :
F < E  ? partial_ordering::greater :
         partial_ordering::unordered
```

except that `E` and `F` are only evaluated once.

(6.4) — Otherwise, `compare_partial_order_fallback(E, F)` is ill-formed.

Change 17.13.1 [coroutine.syn]:

```
namespace std {
  [...]
  // 17.13.5 noop coroutine
  noop_coroutine_handle noop_coroutine() noexcept;

  // 17.13.3.6 comparison operators:
  constexpr bool operator==(coroutine_handle<> x, coroutine_handle<> y) noexcept;
- constexpr bool operator!=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
- constexpr bool operator<(coroutine_handle<> x, coroutine_handle<> y) noexcept;
- constexpr bool operator>(coroutine_handle<> x, coroutine_handle<> y) noexcept;
- constexpr bool operator<=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
- constexpr bool operator>=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
+ constexpr strong_ordering operator<=>(coroutine_handle x, coroutine_handle y) noexcept;

  // 17.13.6 trivial awaitables
  [...]
}
```

Replace the `<` in 17.13.3.6 [coroutine.handle.compare] with the new `<=>`:

```
constexpr bool operator==(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

1 *Returns*: `x.address() == y.address()`.

~~constexpr bool operator<(coroutine_handle<> x, coroutine_handle<> y) noexcept;~~

~~*Returns:* less<>()(x.address(), y.address()).~~

constexpr strong_ordering operator<=>(coroutine_handle x, coroutine_handle y) noexcept;

Returns: compare_three_way()(x.address(), y.address()).

§ 5.3 Clause 18: Concepts Library

No changes.

§ 5.4 Clause 19: Diagnostics Library

Changed operators for: `error_category`, `error_code`, and `error_condition`

Change 19.5.1 [system.error.syn]:

```
namespace std {
  [...]
  // [syserr.compare], comparison functions
  bool operator==(const error_code& lhs, const error_code& rhs) noexcept;
  bool operator==(const error_code& lhs, const error_condition& rhs) noexcept;
- bool operator==(const error_condition& lhs, const error_code& rhs) noexcept;
  bool operator==(const error_condition& lhs, const error_condition& rhs) noexcept;
- bool operator!=(const error_code& lhs, const error_code& rhs) noexcept;
- bool operator!=(const error_code& lhs, const error_condition& rhs) noexcept;
- bool operator!=(const error_condition& lhs, const error_code& rhs) noexcept;
- bool operator!=(const error_condition& lhs, const error_condition& rhs) noexcept;
- bool operator< (const error_code& lhs, const error_code& rhs) noexcept;
- bool operator< (const error_condition& lhs, const error_condition& rhs) noexcept;
+ strong_ordering operator<=>(const error_code& lhs, const error_code& rhs) noexcept;
+ strong_ordering operator<=>(const error_condition& lhs, const error_condition& rhs) noexcept;
  [...]
}
```

Change 19.5.2.1 [syserr.errcat.overview]:

```
namespace std {
  class error_category {
  public:
    [...]

    bool operator==(const error_category& rhs) const noexcept;
-   bool operator!=(const error_category& rhs) const noexcept;
-   bool operator< (const error_category& rhs) const noexcept;
+   strong_ordering operator<=>(const error_category& rhs) const noexcept;
  };

  const error_category& generic_category() noexcept;
  const error_category& system_category() noexcept;
}
```

Change 19.5.2.3 [syserr.errcat.nonvirtuals]:

bool operator==(const error_category& rhs) const noexcept;

Returns: this == &rhs.

~~bool operator!=(const error_category& rhs) const noexcept;~~

~~*Returns:* !(*this == rhs).~~

~~bool operator<(const error_category& rhs) const noexcept;~~

~~*Returns:* less<const error_category*>()(this, &rhs).~~

strong_ordering operator<=>(const error_category& rhs) const noexcept;

Returns: compare_three_way()(this, &rhs).

[*Note*: ~~less~~ compare_three_way ([cmp.object]) provides a total ordering for pointers. —*end note*]

Change 19.5.5 [syserr.compare]:

bool operator==(const error_code& lhs, const error_code& rhs) noexcept;

Returns: lhs.category() == rhs.category() &&
lhs.value() == rhs.value()

```
bool operator==(const error_code& lhs, const error_condition& rhs) noexcept;
```

Returns: `lhs.category().equivalent(lhs.value(), rhs) || rhs.category().equivalent(lhs, rhs.value())`

~~`bool operator==(const error_condition& lhs, const error_code& rhs) noexcept;`~~

~~*Returns:* `rhs.category().equivalent(rhs.value(), lhs) || lhs.category().equivalent(rhs, lhs.value())`~~

```
bool operator==(const error_condition& lhs, const error_condition& rhs) noexcept;
```

Returns: `lhs.category() == rhs.category() && lhs.value() == rhs.value()`

~~`bool operator!=(const error_code& lhs, const error_code& rhs) noexcept;`~~
~~`bool operator!=(const error_code& lhs, const error_condition& rhs) noexcept;`~~
~~`bool operator!=(const error_condition& lhs, const error_code& rhs) noexcept;`~~
~~`bool operator!=(const error_condition& lhs, const error_condition& rhs) noexcept;`~~

~~*Returns:* `!(lhs == rhs)`.~~

~~`bool operator<(const error_code& lhs, const error_code& rhs) noexcept;`~~

~~*Returns:* `lhs.category() < rhs.category() || (lhs.category() == rhs.category() && lhs.value() < rhs.value())`~~

~~`bool operator<(const error_condition& lhs, const error_condition& rhs) noexcept;`~~

~~*Returns:* `lhs.category() < rhs.category() || (lhs.category() == rhs.category() && lhs.value() < rhs.value())`~~

```
strong_ordering operator<=>(const error_code& lhs, const error_code& rhs) noexcept;
```

Effects: Equivalent to:

```
if (auto c = lhs.category() <=> rhs.category(); c != 0) return c;
return lhs.value() <=> rhs.value();
```

```
strong_ordering operator<=>(const error_condition& lhs, const error_condition& rhs) noexcept;
```

Effects: Equivalent to:

```
if (auto c = lhs.category() <=> rhs.category(); c != 0) return c;
return lhs.value() <=> rhs.value();
```

§ 5.5 Clause 20: General utilities library

Changed operators for: `pair`, `tuple`, `optional`, `variant`, `monostate`, `bitset`, `allocator`, `unique_ptr`, `shared_ptr`, `memory_resource` `polymorphic_allocator`, `scoped_allocator_adaptor`, `function`, `type_index`.

Change 20.2.1 [utility.syn]:

```
#include <initializer_list>     // see [initializer.list.syn]

namespace std {
  [...]

  // [pairs], class template pair
  template<class T1, class T2>
    struct pair;

- // [pairs.spec], pair specialized algorithms
- template<class T1, class T2>
-   constexpr bool operator==(const pair<T1, T2>&, const pair<T1, T2>&);
- template<class T1, class T2>
-   constexpr bool operator!=(const pair<T1, T2>&, const pair<T1, T2>&);
- template<class T1, class T2>
-   constexpr bool operator< (const pair<T1, T2>&, const pair<T1, T2>&);
- template<class T1, class T2>
-   constexpr bool operator> (const pair<T1, T2>&, const pair<T1, T2>&);
- template<class T1, class T2>
-   constexpr bool operator<=(const pair<T1, T2>&, const pair<T1, T2>&);
- template<class T1, class T2>
-   constexpr bool operator>=(const pair<T1, T2>&, const pair<T1, T2>&);

  template<class T1, class T2>
    constexpr void swap(pair<T1, T2>& x, pair<T1, T2>& y) noexcept(noexcept(x.swap(y)));
```

```
  template<class T1, class T2>
    constexpr see below make_pair(T1&&, T2&&);
}
```

Change 20.4.2 [pairs.pair]:

```
namespace std {
  template<class T1, class T2>
  struct pair {
    using first_type  = T1;
    using second_type = T2;

    T1 first;
    T2 second;

    [...]

    constexpr void swap(pair& p) noexcept(see below);

+   // 20.4.3, pair specialized algorithms
+   friend constexpr bool operator==(const pair&, const pair&) = default;
+   friend constexpr common_comparison_category_t<synth-3way-result<T1>, synth-3way-result<T2>>
+     operator<=>(const pair&, const pair&)
+     { see below }
  };

  template<class T1, class T2>
    pair(T1, T2) -> pair<T1, T2>;
}
```

Change 20.4.3 [pairs.spec]:

```
template<class T1, class T2>
  constexpr bool operator==(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

1 *Returns:* ~~`x.first == y.first && x.second == y.second`~~.

```
template<class T1, class T2>
  constexpr bool operator!=(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

2 ~~*Returns:* `!(x == y)`.~~

```
template<class T1, class T2>
  constexpr bool operator<(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

3 ~~*Returns:* `x.first < y.first || (!(y.first < x.first) && x.second < y.second)`.~~

```
template<class T1, class T2>
  constexpr bool operator>(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

4 ~~*Returns:* `y < x`.~~

```
template<class T1, class T2>
  constexpr bool operator<=(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

5 ~~*Returns:* `!(y < x)`.~~

```
template<class T1, class T2>
  constexpr bool operator>=(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

6 ~~*Returns:* `!(x < y)`.~~

```
friend constexpr common_comparison_category_t<synth-3way-result<T1>, synth-3way-result<T2>>
  operator<=>(const pair&, const pair&);
```

7 *Effects*: Equivalent to:

```
if (auto c = synth-3way(lhs.first, rhs.first); c != 0) return c;
return synth-3way(lhs.second, rhs.second);
```

Change 20.5.3 [tuple.syn]:

```
namespace std {
  // [tuple.tuple], class template tuple
  template<class... Types>
    class tuple;

  [...]

  // [tuple.rel], relational operators
```

```
  template<class... TTypes, class... UTypes>
    constexpr bool operator==(const tuple<TTypes...>&, const tuple<UTypes...>&);
- template<class... TTypes, class... UTypes>
-   constexpr bool operator!=(const tuple<TTypes...>&, const tuple<UTypes...>&);
- template<class... TTypes, class... UTypes>
-   constexpr bool operator<(const tuple<TTypes...>&, const tuple<UTypes...>&);
- template<class... TTypes, class... UTypes>
-   constexpr bool operator>(const tuple<TTypes...>&, const tuple<UTypes...>&);
- template<class... TTypes, class... UTypes>
-   constexpr bool operator<=(const tuple<TTypes...>&, const tuple<UTypes...>&);
- template<class... TTypes, class... UTypes>
-   constexpr bool operator>=(const tuple<TTypes...>&, const tuple<UTypes...>&);
+ template<class... TTypes, class... UTypes>
+   constexpr common_comparison_category_t<synth-3way-result<TTypes, UTypes>...>
+     operator<=>(const tuple<TTypes...>&, const tuple<UTypes...>&);

  [...]
}
```

Change 20.5.3.8 [tuple.rel]:

```
template<class... TTypes, class... UTypes>
  constexpr bool operator==(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

1 *Requires*: For all `i`, where `0 <= i` and `i < sizeof...(TTypes)`, `get<i>(t) == get<i>(u)` is a valid expression returning a type that is convertible to `bool`. `sizeof...(TTypes) == sizeof...(UTypes)`.

2 *Returns*: `true` if `get<i>(t) == get<i>(u)` for all `i`, otherwise `false`. For any two zero-length tuples `e` and `f`, `e == f` returns `true`.

3 *Effects*: The elementary comparisons are performed in order from the zeroth index upwards. No comparisons or element accesses are performed after the first equality comparison that evaluates to `false`.

```
~~template<class... TTypes, class... UTypes>~~
~~  constexpr bool operator!=(const tuple<TTypes...>& t, const tuple<UTypes...>& u);~~
```

4 ~~*Returns:* `!(t == u)`.~~

```
~~template<class... TTypes, class... UTypes>~~
~~  constexpr bool operator<(const tuple<TTypes...>& t, const tuple<UTypes...>& u);~~
```

5 ~~*Requires:* For all `i`, where `0 <= i` and `i < sizeof...(TTypes)`, both `get<i>(t)` and `get<i>(u)` are valid expressions returning types that are convertible to `bool`. `sizeof...(TTypes) == sizeof...(UTypes)`.~~

6 ~~*Returns:* The result of a lexicographical comparison between `t` and `u`. The result is defined as: `(bool(get<0>(t) < get<0>(u)) || (bool(get<0>(t) == get<0>(u)) && t_tail < u_tail)`, where `r_tail` for some tuple `r` is a tuple containing all but the first element of `r`. For any two zero-length tuples `e` and `f`, `e < f` returns `false`.~~

```
~~template<class... TTypes, class... UTypes>~~
~~  constexpr bool operator>(const tuple<TTypes...>& t, const tuple<UTypes...>& u);~~
```

7 ~~*Returns:* `u < t`.~~

```
~~template<class... TTypes, class... UTypes>~~
~~  constexpr bool operator<=(const tuple<TTypes...>& t, const tuple<UTypes...>& u);~~
```

8 ~~*Returns:* `!(u < t)`.~~

```
~~template<class... TTypes, class... UTypes>~~
~~  constexpr bool operator>=(const tuple<TTypes...>& t, const tuple<UTypes...>& u);~~
```

9 ~~*Returns:* `!(t < u)`.~~

```
template<class... TTypes, class... UTypes>
  constexpr common_comparison_category_t<synth-3way-result<TTypes, UTypes>...>
    operator<=>(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

10 *Requires*: For all `i`, where `0 <= i` and `i < sizeof...(TTypes)`, both *synth-3way*`(get<i>(t), get<i>(u))` is a valid expression. `sizeof...(TTypes) == sizeof...(UTypes)`.

11 *Effects*: Performs a lexicographical comparison between `t` and `u`. For any two zero-length tuples `t` and `u`, `t <=> u` returns `strong_ordering::equal`. Otherwise, equivalent to:

```
auto c = synth-3way(get<0>(t), get<0>(u));
return (c != 0) ? c : (t_tail <=> u_tail);
```

where r_{tail} for some tuple `r` is a tuple containing all but the first element of `r`.

12 [*Note*: The above ~~definitions for comparison functions do not require~~ definition does not require t_{tail} (or u_{tail}) to be constructed. It may not even be possible, as `t` and `u` are not required to be copy constructible. Also, all comparison functions are short circuited; they do not perform element accesses beyond what is required to determine the result of the comparison. —*end note*]

Change 20.6.2 [optional.syn]:

```
namespace std {
  // [optional.optional], class template optional
  template<class T>
    class optional;

  // [optional.nullopt], no-value state indicator
  struct nullopt_t{see below};
  inline constexpr nullopt_t nullopt(unspecified);

  // [optional.bad.access], class bad_optional_access
  class bad_optional_access;

  // [optional.relops], relational operators
  template<class T, class U>
    constexpr bool operator==(const optional<T>&, const optional<U>&);
  template<class T, class U>
    constexpr bool operator!=(const optional<T>&, const optional<U>&);
  template<class T, class U>
    constexpr bool operator<(const optional<T>&, const optional<U>&);
  template<class T, class U>
    constexpr bool operator>(const optional<T>&, const optional<U>&);
  template<class T, class U>
    constexpr bool operator<=(const optional<T>&, const optional<U>&);
  template<class T, class U>
    constexpr bool operator>=(const optional<T>&, const optional<U>&);
+ template<class T, ThreeWayComparableWith<T> U>
+   constexpr compare_three_way_result_t<T,U>
+     operator<=>(const optional<T>&, const optional<U>&);

  // [optional.nullops], comparison with nullopt
  template<class T> constexpr bool operator==(const optional<T>&, nullopt_t) noexcept;
- template<class T> constexpr bool operator==(nullopt_t, const optional<T>&) noexcept;
- template<class T> constexpr bool operator!=(const optional<T>&, nullopt_t) noexcept;
- template<class T> constexpr bool operator!=(nullopt_t, const optional<T>&) noexcept;
- template<class T> constexpr bool operator<(const optional<T>&, nullopt_t) noexcept;
- template<class T> constexpr bool operator<(nullopt_t, const optional<T>&) noexcept;
- template<class T> constexpr bool operator>(const optional<T>&, nullopt_t) noexcept;
- template<class T> constexpr bool operator>(nullopt_t, const optional<T>&) noexcept;
- template<class T> constexpr bool operator<=(const optional<T>&, nullopt_t) noexcept;
- template<class T> constexpr bool operator<=(nullopt_t, const optional<T>&) noexcept;
- template<class T> constexpr bool operator>=(const optional<T>&, nullopt_t) noexcept;
- template<class T> constexpr bool operator>=(nullopt_t, const optional<T>&) noexcept;
+ template<class T> constexpr strong_ordering operator<=>(const optional<T>&, nullopt_t) noexcept;

  // [optional.comp.with.t], comparison with T
  template<class T, class U> constexpr bool operator==(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator==(const T&, const optional<U>&);
  template<class T, class U> constexpr bool operator!=(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator!=(const T&, const optional<U>&);
  template<class T, class U> constexpr bool operator<(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator<(const T&, const optional<U>&);
  template<class T, class U> constexpr bool operator>(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator>(const T&, const optional<U>&);
  template<class T, class U> constexpr bool operator<=(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator<=(const T&, const optional<U>&);
  template<class T, class U> constexpr bool operator>=(const optional<T>&, const U&);
  template<class T, class U> constexpr bool operator>=(const T&, const optional<U>&);
+ template<class T, ThreeWayComparableWith<T> U>
+   constexpr compare_three_way_result_t<T,U>
+     operator<=>(const optional<T>&, const U&);

  // [optional.specalg], specialized algorithms
  template<class T>
    void swap(optional<T>&, optional<T>&) noexcept(see below);

  [...]
}
```

Add to 20.6.6 [optional.relops]:

```
template<class T, class U> constexpr bool operator>=(const optional<T>& x, const optional<U>& y);
```

16 *Requires*: The expression `*x >= *y` shall be well-formed and its result shall be convertible to `bool`.

17 *Returns*: If `!y`, `true`; otherwise, if `!x`, `false`; otherwise `*x >= *y`.

18 *Remarks*: Specializations of this function template for which `*x >= *y` is a core constant expression shall be constexpr functions.

```
template<class T, ThreeWayComparableWith<T> U>
  constexpr compare_three_way_result_t<T,U>
    operator<=>(const optional<T>& x, const optional<U>& y);
```

19 *Returns*: If `x && y`, `*x <=> *y`; otherwise `bool(x) <=> bool(y)`.

20 *Remarks*: Specializations of this function template for which `*x <=> *y` is a core constant expression shall be constexpr functions.

Change 20.6.7 [optional.nullops], removing most of the comparisons:

```
template<class T> constexpr bool operator==(const optional<T>& x, nullopt_t) noexcept;
~~template<class T> constexpr bool operator==(nullopt_t, const optional<T>& x) noexcept;~~
```

1 *Returns*: `!x`.

```
template<class T> constexpr strong_ordering operator<=>(const optional<T>& x, nullopt_t) noexcept;
```

2 *Returns*: `bool(x) <=> false`.

```
~~template<class T> constexpr bool operator!=(const optional<T>& x, nullopt_t) noexcept;~~
~~template<class T> constexpr bool operator!=(nullopt_t, const optional<T>& x) noexcept;~~
```

2 ~~*Returns:* `bool(x)`.~~

```
~~template<class T> constexpr bool operator<(const optional<T>& x, nullopt_t) noexcept;~~
```

3 ~~*Returns:* `false`.~~

```
~~template<class T> constexpr bool operator<(nullopt_t, const optional<T>& x) noexcept;~~
```

4 ~~*Returns:* `bool(x)`.~~

```
~~template<class T> constexpr bool operator>(const optional<T>& x, nullopt_t) noexcept;~~
```

5 ~~*Returns:* `bool(x)`.~~

```
~~template<class T> constexpr bool operator>(nullopt_t, const optional<T>& x) noexcept;~~
```

6 ~~*Returns:* `false`.~~

```
~~template<class T> constexpr bool operator<=(const optional<T>& x, nullopt_t) noexcept;~~
```

7 ~~*Returns:* `!x`.~~

```
~~template<class T> constexpr bool operator<=(nullopt_t, const optional<T>& x) noexcept;~~
```

8 ~~*Returns:* `true`.~~

```
~~template<class T> constexpr bool operator>=(const optional<T>& x, nullopt_t) noexcept;~~
```

9 ~~*Returns:* `true`.~~

```
~~template<class T> constexpr bool operator>=(nullopt_t, const optional<T>& x) noexcept;~~
```

10 ~~*Returns:* `!x`.~~

Add to 20.6.8 [optional.comp.with.t]:

```
template<class T, class U> constexpr bool operator>=(const T& v, const optional<U>& x);
```

23 *Requires*: The expression `v >= *x` shall be well-formed and its result shall be convertible to `bool`.

24 *Effects*: Equivalent to: `return bool(x) ? v >= *x : true;`

```
template<class T, ThreeWayComparableWith<T> U>
  constexpr compare_three_way_result_t<T,U>
    operator<=>(const optional<T>& x, const U& v);
```

25 *Effects*: Equivalent to: `return bool(x) ? *x <=> v : strong_ordering::less;`

Change 20.7.2 [variant.syn]:

```
namespace std {
  // [variant.variant], class template variant
  template<class... Types>
    class variant;

  [...]

  // [variant.relops], relational operators
  template<class... Types>
    constexpr bool operator==(const variant<Types...>&, const variant<Types...>&);
  template<class... Types>
    constexpr bool operator!=(const variant<Types...>&, const variant<Types...>&);
  template<class... Types>
    constexpr bool operator<(const variant<Types...>&, const variant<Types...>&);
  template<class... Types>
    constexpr bool operator>(const variant<Types...>&, const variant<Types...>&);
  template<class... Types>
    constexpr bool operator<=(const variant<Types...>&, const variant<Types...>&);
  template<class... Types>
    constexpr bool operator>=(const variant<Types...>&, const variant<Types...>&);
+ template<class... Types> requires (ThreeWayComparable<Types> && ...)
+   constexpr common_comparison_category_t<compare_three_way_result_t<Types>...>
+     operator<=>(const variant<Types...>&, const variant<Types...>&);

  // [variant.visit], visitation
  template<class Visitor, class... Variants>
    constexpr see below visit(Visitor&&, Variants&&...);
  template<class R, class Visitor, class... Variants>
    constexpr R visit(Visitor&&, Variants&&...);

  // [variant.monostate], class monostate
  struct monostate;

  // [variant.monostate.relops], monostate relational operators
  constexpr bool operator==(monostate, monostate) noexcept;
- constexpr bool operator!=(monostate, monostate) noexcept;
- constexpr bool operator<(monostate, monostate) noexcept;
- constexpr bool operator>(monostate, monostate) noexcept;
- constexpr bool operator<=(monostate, monostate) noexcept;
- constexpr bool operator>=(monostate, monostate) noexcept;
+ constexpr strong_ordering operator<=>(monostate, monostate) noexcept;

  [...]
}
```

Add to 20.7.6 [variant.relops]:

```
template<class... Types>
  constexpr bool operator>=(const variant<Types...>& v, const variant<Types...>& w);
```

11 *Requires*: `get<i>(v) >= get<i>(w)` is a valid expression returning a type that is convertible to `bool`, for all `i`.

12 *Returns*: If `w.valueless_by_exception()`, `true`; otherwise if `v.valueless_by_exception()`, `false`; otherwise, if `v.index() > w.index()`, `true`; otherwise if `v.index() < w.index()`, `false`; otherwise `get<i>(v) >= get<i>(w)` with `i` being `v.index()`.

```
template<class... Types> requires (ThreeWayComparable<Types> && ...)
  constexpr common_comparison_category_t<compare_three_way_result_t<Types>...>
    operator<=>(const variant<Types...>& v, const variant<Types...>& w);
```

13 *Returns*: Let `c` be `(v.index() + 1) <=> (w.index() + 1)`. If `c != 0`, `c`. Otherwise, `get<i>(v) <=> get<i>(w)` with `i` being `v.index()`.

Simplify 20.7.9 [variant.monostate.relops]:

```
constexpr bool operator==(monostate, monostate) noexcept { return true; }
```

```
~~constexpr bool operator!=(monostate, monostate) noexcept { return false; }~~
~~constexpr bool operator<(monostate, monostate) noexcept { return false; }~~
~~constexpr bool operator>(monostate, monostate) noexcept { return false; }~~
~~constexpr bool operator<=(monostate, monostate) noexcept { return true; }~~
~~constexpr bool operator>=(monostate, monostate) noexcept { return true; }~~

constexpr strong_ordering operator<=>(monostate, monostate) noexcept { return strong_ordering::equal; }
```

1 [*Note*: monostate objects have only a single state; they thus always compare equal. —*end note*]

Change 20.9.2 [template.bitset]:

```
namespace std {
  template<size_t N> class bitset {
  public:
    [...]
    size_t count() const noexcept;
    constexpr size_t size() const noexcept;
    bool operator==(const bitset<N>& rhs) const noexcept;
-   bool operator!=(const bitset<N>& rhs) const noexcept;
    bool test(size_t pos) const;
    [...]
  };
}
```

Change 20.9.2.2 [bitset.members]:

```
bool operator==(const bitset<N>& rhs) const noexcept;
```

36 *Returns*: `true` if the value of each bit in `*this` equals the value of the corresponding bit in `rhs`.

```
~~bool operator!=(const bitset<N>& rhs) const noexcept;~~
```

37 ~~*Returns:* `true` if `!(*this == rhs)`.~~

Change 20.10.2 [memory.syn]:

```
namespace std{
  [...]

  // [default.allocator], the default allocator
  template<class T> class allocator;
  template<class T, class U>
    bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
- template<class T, class U>
-   bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;

  [...]

  // [unique.ptr], class template unique_ptr
  [...]
  template<class T, class D>
    void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;

  template<class T1, class D1, class T2, class D2>
    bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
- template<class T1, class D1, class T2, class D2>
-   bool operator!=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
  template<class T1, class D1, class T2, class D2>
    bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
  template<class T1, class D1, class T2, class D2>
    bool operator>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
  template<class T1, class D1, class T2, class D2>
    bool operator<=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
  template<class T1, class D1, class T2, class D2>
    bool operator>=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
+ template<class T1, class D1, class T2, class D2>
+     requires ThreeWayComparableWith<typename unique_ptr<T1, D1>::pointer,
+                                     typename unique_ptr<T2, D2>::pointer>
+   compare_three_way_result_t<typename unique_ptr<T1, D1>::pointer, typename unique_ptr<T2,
D2>::pointer>
+     operator<=>(const unique_ptr& x, const unique_ptr<T2, D2>& y);

  template<class T, class D>
    bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept;
- template<class T, class D>
-   bool operator==(nullptr_t, const unique_ptr<T, D>& y) noexcept;
- template<class T, class D>
-   bool operator!=(const unique_ptr<T, D>& x, nullptr_t) noexcept;
- template<class T, class D>
-   bool operator!=(nullptr_t, const unique_ptr<T, D>& y) noexcept;
  template<class T, class D>
```

```
    bool operator<(const unique_ptr<T, D>& x, nullptr_t);
  template<class T, class D>
    bool operator<(nullptr_t, const unique_ptr<T, D>& y);
  template<class T, class D>
    bool operator>(const unique_ptr<T, D>& x, nullptr_t);
  template<class T, class D>
    bool operator>(nullptr_t, const unique_ptr<T, D>& y);
  template<class T, class D>
    bool operator<=(const unique_ptr<T, D>& x, nullptr_t);
  template<class T, class D>
    bool operator<=(nullptr_t, const unique_ptr<T, D>& y);
  template<class T, class D>
    bool operator>=(const unique_ptr<T, D>& x, nullptr_t);
  template<class T, class D>
    bool operator>=(nullptr_t, const unique_ptr<T, D>& y);
+ template<class T, class D>
+     requires ThreeWayComparableWith<typename unique_ptr<T, D>::pointer, nullptr_t>
+   compare_three_way_result_t<typename unique_ptr<T, D>::pointer, nullptr_t>
+     operator<=>(const unique_ptr<T, D>& x, nullptr_t);

  template<class E, class T, class Y, class D>
    basic_ostream<E, T>& operator<<(basic_ostream<E, T>& os, const unique_ptr<Y, D>& p);

  [...]


  // [util.smartptr.shared.cmp], shared_ptr comparisons
  template<class T, class U>
    bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
- template<class T, class U>
-   bool operator!=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
- template<class T, class U>
-   bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
- template<class T, class U>
-   bool operator>(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
- template<class T, class U>
-   bool operator<=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
- template<class T, class U>
-   bool operator>=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
+ template<class T, class U>
+   strong_ordering operator<=>(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

  template<class T>
    bool operator==(const shared_ptr<T>& x, nullptr_t) noexcept;
- template<class T>
-   bool operator==(nullptr_t, const shared_ptr<T>& y) noexcept;
- template<class T>
-   bool operator!=(const shared_ptr<T>& x, nullptr_t) noexcept;
- template<class T>
-   bool operator!=(nullptr_t, const shared_ptr<T>& y) noexcept;
- template<class T>
-   bool operator<(const shared_ptr<T>& x, nullptr_t) noexcept;
- template<class T>
-   bool operator<(nullptr_t, const shared_ptr<T>& y) noexcept;
- template<class T>
-   bool operator>(const shared_ptr<T>& x, nullptr_t) noexcept;
- template<class T>
-   bool operator>(nullptr_t, const shared_ptr<T>& y) noexcept;
- template<class T>
-   bool operator<=(const shared_ptr<T>& x, nullptr_t) noexcept;
- template<class T>
-   bool operator<=(nullptr_t, const shared_ptr<T>& y) noexcept;
- template<class T>
-   bool operator>=(const shared_ptr<T>& x, nullptr_t) noexcept;
- template<class T>
-   bool operator>=(nullptr_t, const shared_ptr<T>& y) noexcept;
+ template<class T>
+   strong_ordering operator<=>(const shared_ptr<T>& x, nullptr_t) noexcept;

  [...]
}
```

Remove from 20.10.10.2 [allocator.globals]:

```
template<class T, class U>
  bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
```

1 *Returns*: `true`.

```
~~template<class T, class U>~~
~~  bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;~~
```

2 ~~*Returns:*~~ ~~`false`~~.

Change 20.11.1.5 [unique.ptr.special]:

```
template<class T1, class D1, class T2, class D2>
  bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

Returns: x .get () == y .get ().

```
template<class T1, class D1, class T2, class D2>
  bool operator!=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

Returns: x .get () != y .get ().

[…]

```
template<class T1, class D1, class T2, class D2>
  bool operator>=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

Returns: !(x < y).

```
template<class T1, class D1, class T2, class D2>
    requires ThreeWayComparableWith<typename unique_ptr<T1, D1>::pointer,
                                    typename unique_ptr<T2, D2>::pointer>
  compare_three_way_result_t<typename unique_ptr<T1, D1>::pointer, typename unique_ptr<T2, D2>::pointer>
    operator<=>(const unique_ptr& x, const unique_ptr<T2, D2>& y);
```

Returns: compare_three_way ()(x .get (), y .get ()).

```
template<class T, class D>
  bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template<class T, class D>
bool operator==(nullptr_t, const unique_ptr<T, D>& x) noexcept;
```

Returns: !x.

```
template<class T, class D>
  bool operator!=(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template<class T, class D>
  bool operator!=(nullptr_t, const unique_ptr<T, D>& x) noexcept;
```

Returns: x bool).

[…]

```
template<class T, class D>
  bool operator>=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
  bool operator>=(nullptr_t, const unique_ptr<T, D>& x);
```

Returns: The first function template returns !(x < nullptr). The second function template returns !(nullptr < x).

```
template<class T, class D>
    requires ThreeWayComparableWith<typename unique_ptr<T, D>::pointer, nullptr_t>
  compare_three_way_result_t<typename unique_ptr<T, D>::pointer, nullptr_t>
    operator<=>(const unique_ptr<T, D>& x, nullptr_t);
```

Returns: compare_three_way ()(x .get (), nullptr).

Change 20.11.3.7 [util.smartptr.shared.cmp]:

```
template<class T, class U>
  bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
```

Returns: a .get () == b .get ().

```
template<class T, class U>
  bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
```

Returns: less <>()(a .get (), b .get ()).

[*Note*: Defining a comparison function allows shared_ptr objects to be used as keys in associative containers. — *end note*]

```
template<class T>
  bool operator==(const shared_ptr<T>& a, nullptr_t) noexcept;
template<class T>
bool operator==(nullptr_t, const shared_ptr<T>& a) noexcept;
```

Returns: !a.

```
template<class T>
  bool operator!=(const shared_ptr<T>& a, nullptr_t) noexcept;
```

```
template<class T>
  bool operator!=(nullptr_t, const shared_ptr<T>& a) noexcept;
```

Returns: `!(a == nullptr)`.

```
template<class T>
  bool operator<(const shared_ptr<T>& a, nullptr_t) noexcept;
template<class T>
  bool operator<(nullptr_t, const shared_ptr<T>& a) noexcept;
```

Returns: The first function template returns `less<typename shared_ptr<T>::element_type*>()(a.get(), nullptr)`. The second function template returns `less<typename shared_ptr<T>::element_type*>()(nullptr, a.get())`.

```
template<class T>
  bool operator>(const shared_ptr<T>& a, nullptr_t) noexcept;
template<class T>
  bool operator>(nullptr_t, const shared_ptr<T>& a) noexcept;
```

Returns: The first function template returns `nullptr < a`. The second function template returns `a <= nullptr`.

```
template<class T>
  bool operator<=(const shared_ptr<T>& a, nullptr_t) noexcept;
template<class T>
  bool operator<=(nullptr_t, const shared_ptr<T>& a) noexcept;
```

Returns: The first function template returns `!(nullptr < a)`. The second function template returns `!(a < nullptr)`.

```
template<class T>
  bool operator>=(const shared_ptr<T>& a, nullptr_t) noexcept;
template<class T>
  bool operator>=(nullptr_t, const shared_ptr<T>& a) noexcept;
```

Returns: The first function template returns `!(a < nullptr)`. The second function template returns `!(nullptr < a)`.

```
template<class T, class U>
  strong_ordering operator<=>(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
```

Returns: `compare_three_way()(a.get(), b.get())`.

[*Note*: Defining a comparison function allows `shared_ptr` objects to be used as keys in associative containers. —*end note*]

```
template<class T>
  strong_ordering operator<=>(const shared_ptr<T>& a, nullptr_t) noexcept;
```

Returns: `compare_three_way()(a.get(), nullptr)`.

Change 20.12.1 [mem.res.syn]:

```
namespace std::pmr {
  // [mem.res.class], class memory_resource
  class memory_resource;

  bool operator==(const memory_resource& a, const memory_resource& b) noexcept;
- bool operator!=(const memory_resource& a, const memory_resource& b) noexcept;

  // [mem.poly.allocator.class], class template polymorphic_allocator
  template<class Tp> class polymorphic_allocator;

  template<class T1, class T2>
    bool operator==(const polymorphic_allocator<T1>& a,
                    const polymorphic_allocator<T2>& b) noexcept;
- template<class T1, class T2>
-   bool operator!=(const polymorphic_allocator<T1>& a,
-                   const polymorphic_allocator<T2>& b) noexcept;

  [...]
}
```

Change 20.12.2.3 [mem.res.eq]:

```
bool operator==(const memory_resource& a, const memory_resource& b) noexcept;
```

Returns: `&a == &b || a.is_equal(b)`.

```
bool operator!=(const memory_resource& a, const memory_resource& b) noexcept;
```

2 ~~*Returns:* `!(a == b)`.~~

Change 20.12.3.3 [mem.poly.allocator.eq]:

```
template<class T1, class T2>
  bool operator==(const polymorphic_allocator<T1>& a,
                  const polymorphic_allocator<T2>& b) noexcept;
```

1 *Returns*: `*a.resource() == *b.resource()`.

```
~~template<class T1, class T2>~~
~~  bool operator!=(const polymorphic_allocator<T1>& a,~~
~~                  const polymorphic_allocator<T2>& b) noexcept;~~
```

2 ~~*Returns:* `!(a == b)`.~~

Change 20.13.1 [allocator.adaptor.syn]:

```
namespace std {
  // class template scoped allocator adaptor
  template<class OuterAlloc, class... InnerAlloc>
    class scoped_allocator_adaptor;

  // [scoped.adaptor.operators], scoped allocator operators
  template<class OuterA1, class OuterA2, class... InnerAllocs>
    bool operator==(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                    const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
- template<class OuterA1, class OuterA2, class... InnerAllocs>
-   bool operator!=(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
-                   const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
}
```

Change 20.13.5 [scoped.adaptor.operators]:

```
template<class OuterA1, class OuterA2, class... InnerAllocs>
  bool operator==(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                  const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
```

1 *Returns*: If `sizeof...(InnerAllocs)` is zero, `a.outer_allocator() == b.outer_allocator()` otherwise `a.outer_allocator() == b.outer_allocator() && a.inner_allocator() == b.inner_allocator()`

```
~~template<class OuterA1, class OuterA2, class... InnerAllocs>~~
~~  bool operator!=(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,~~
~~                  const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;~~
```

2 ~~*Returns:* `!(a == b)`.~~

Change 20.14.1 [functional.syn]:

```
namespace std {
  [...]
  template<class R, class... ArgTypes>
    void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&) noexcept;

  template<class R, class... ArgTypes>
    bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
- template<class R, class... ArgTypes>
-   bool operator==(nullptr_t, const function<R(ArgTypes...)>&) noexcept;
- template<class R, class... ArgTypes>
-   bool operator!=(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
- template<class R, class... ArgTypes>
-   bool operator!=(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

  // [func.search], searchers
  template<class ForwardIterator, class BinaryPredicate = equal_to<>>
    class default_searcher;
  [...]
}
```

Change 20.14.8 [range.cmp]/2 to add `<=>`:

There is an implementation-defined strict total ordering over all pointer values of a given type. This total ordering is consistent with the partial order imposed by the builtin operators `<`, `>`, `<=`, ~~and~~ `>=`, and `<=>`.

Change 20.14.16.2 [func.wrap.func]:

```
namespace std {
  template<class> class function; // not defined
```

```
  [...]

  // [func.wrap.func.nullptr], Null pointer comparisons
  template<class R, class... ArgTypes>
    bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;

- template<class R, class... ArgTypes>
-   bool operator==(nullptr_t, const function<R(ArgTypes...)>&) noexcept;
-
- template<class R, class... ArgTypes>
-   bool operator!=(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
-
- template<class R, class... ArgTypes>
-   bool operator!=(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

  [...]
}
```

Change 20.14.16.2.6 [func.wrap.func.nullptr]:

```
template<class R, class... ArgTypes>
  bool operator==(const function<R(ArgTypes...)>& f, nullptr_t) noexcept;
~~template<class R, class… ArgTypes>~~
~~bool operator==(nullptr_t, const function<R(ArgTypes…)>& f) noexcept;~~
```

1 *Returns*: `!f`.

```
~~template<class R, class... ArgTypes>~~
~~  bool operator!=(const function<R(ArgTypes...)>& f, nullptr_t) noexcept;~~
~~template<class R, class... ArgTypes>~~
~~  bool operator!=(nullptr_t, const function<R(ArgTypes...)>& f) noexcept;~~
```

2 ~~*Returns:* `f` `bool` `.`~~

Add a new row to 20.15.4.3 [meta.unary.prop], the “Type property predicates” table:

Template	Condition	Preconditions>
`template<class T> struct has_strong_structural_equality;`	The type `T` has strong structural equality ([class.compare.default]).	`T` shall be a complete type, *cv* `void`, or an array of unknown bound.

Change 20.15.7.6 [meta.trans.other] to add special rule for comparison categories:

3 Note A: For the `common_type` trait applied to a template parameter pack `T` of types, the member `type` shall be either defined or not present as follows:

(3.1) — If `sizeof...(T)` is zero, there shall be no member `type`.

(3.2) — If `sizeof...(T)` is one, let `T0` denote the sole type constituting the pack `T`. The member typedef-name `type` shall denote the same type, if any, as `common_type_t<T0, T0>`; otherwise there shall be no member `type`.

(3.3) — If `sizeof...(T)` is two, let the first and second types constituting `T` be denoted by `T1` and `T2`, respectively, and let `D1` and `D2` denote the same types as `decay_t<T1>` and `decay_t<T2>`, respectively.

(3.3.1) — If `is_same_v<T1, D1>` is `false` or `is_same_v<T2, D2>` is `false`, let `C` denote the same type, if any, as `common_type_t<D1, D2>`.

(3.3.2) — [*Note*: None of the following will apply if there is a specialization `common_type<D1, D2>`. —*end note*]

(3.3.*) — Otherwise, if both `D1` and `D2` denote comparison category type ([cmp.categories.pre]), let `C` denote the common comparison type ([class.spaceship]) of `D1` and `D2`.

(3.3.3) — Otherwise, if `decay_t<decltype(false ? declval<D1>() : declval<D2>())>` denotes a valid type, let `C` denote that type.

(3.3.4) — Otherwise, if `COND_RES(CREF(D1), CREF(D2))` denotes a type, let `C` denote the type `decay_t<COND_RES(CREF(D1), CREF(D2))>`. In either case, the member typedef-name type shall denote the same type, if any, as `C`. Otherwise, there shall be no member `type`.

(3.4) — If `sizeof...(T)` is greater than two, let `T1`, `T2`, and `R`, respectively, denote the first, second, and (pack of) remaining types constituting `T`. Let `C` denote the same type, if any, as `common_type_t<T1, T2>`. If there is such a type `C`, the member typedef-name `type` shall denote the same type, if any, as `common_type_t<C, R...>`. Otherwise, there shall be no member `type`.

Change 20.17.2 [type.index.overview]. Note that the relational operators on `type_index` are based on `type_info::before` (effectively `<`). `type_info` *could* provide a three-way ordering function, but does not. Since an important motivation for the existence of `type_index` is

to be used as a key in an associative container, we do not want to pessimize `<` - but do want to provide `<=>`.

```
namespace std {
  class type_index {
  public:
    type_index(const type_info& rhs) noexcept;
    bool operator==(const type_index& rhs) const noexcept;
-   bool operator!=(const type_index& rhs) const noexcept;
    bool operator< (const type_index& rhs) const noexcept;
    bool operator> (const type_index& rhs) const noexcept;
    bool operator<=(const type_index& rhs) const noexcept;
    bool operator>=(const type_index& rhs) const noexcept;
+   strong_ordering operator<=>(const type_index& rhs) const noexcept;
    size_t hash_code() const noexcept;
    const char* name() const noexcept;

  private:
    const type_info* target;    // exposition only
    // Note that the use of a pointer here, rather than a reference,
    // means that the default copy/move constructor and assignment
    // operators will be provided and work as expected.
  };
}
```

Change 20.17.3 [type.index.members]:

```
bool operator==(const type_index& rhs) const noexcept;
```

2 *Returns*: `*target == *rhs.target`.

```
~~bool operator!=(const type_index& rhs) const noexcept;~~
```

3 ~~*Returns:* `*target != *rhs.target`.~~

```
bool operator<(const type_index& rhs) const noexcept;
```

4 *Returns*: `target->before(*rhs.target)`.

```
bool operator>(const type_index& rhs) const noexcept;
```

5 *Returns*: `rhs.target->before(*target)`.

```
bool operator<=(const type_index& rhs) const noexcept;
```

6 *Returns*: `!rhs.target->before(*target)`.

```
bool operator>=(const type_index& rhs) const noexcept;
```

7 *Returns*: `!target->before(*rhs.target)`.

```
strong_ordering operator<=>(const type_index& rhs) const noexcept;
```

8 *Effects*: Equivalent to:

```
if (*target == *rhs.target) return strong_ordering::equal;
if (target->before(*rhs.target)) return strong_ordering::less;
return strong_ordering::greater;
```

Change 20.19.1 [charconv.syn]:

```
namespace std {
  [...]

  // [charconv.to.chars], primitive numerical output conversion
  struct to_chars_result {
    char* ptr;
    errc ec;

+   friend bool operator==(const to_chars_result&, const to_chars_result&) = default;
  };

  [...]

  // [charconv.from.chars], primitive numerical input conversion
  struct from_chars_result {
    const char* ptr;
    errc ec;

+   friend bool operator==(const from_chars_result&, const from_chars_result&) = default;
  };
```

```
  [...]
}
```

§ 5.6 Clause 21: Strings library

Changing the operators for `basic_string` and `basic_string_view` and adding extra type alises to the `char_traits` specializations provided by the standard.

Change 21.2.3.1 [char.traits.specializations.char]:

```
namespace std {
  template<> struct char_traits<char> {
    using char_type  = char;
    using int_type   = int;
    using off_type   = streamoff;
    using pos_type   = streampos;
    using state_type = mbstate_t;
+   using comparison_category = strong_ordering;
    [...]
  };
}
```

Change 21.2.3.2 [char.traits.specializations.char8.t]

```
namespace std {
  template<> struct char_traits<char8_t> {
    using char_type = char8_t;
    using int_type = unsigned int;
    using off_type = streamoff;
    using pos_type = u8streampos;
    using state_type = mbstate_t;
+   using comparison_category = strong_ordering;
    [...]
  };
}
```

Change 21.2.3.3 [char.traits.specializations.char16.t]:

```
namespace std {
  template<> struct char_traits<char16_t> {
    using char_type  = char16_t;
    using int_type   = uint_least16_t;
    using off_type   = streamoff;
    using pos_type   = u16streampos;
    using state_type = mbstate_t;
+   using comparison_category = strong_ordering;
    [...]
  };
}
```

Change 21.2.3.4 [char.traits.specializations.char32.t]:

```
namespace std {
  template<> struct char_traits<char32_t> {
    using char_type  = char32_t;
    using int_type   = uint_least32_t;
    using off_type   = streamoff;
    using pos_type   = u32streampos;
    using state_type = mbstate_t;
+   using comparison_category = strong_ordering;
    [...]
  };
}
```

Change 21.2.3.5 [char.traits.specializations.wchar.t]:

```
namespace std {
  template<> struct char_traits<wchar_t> {
    using char_type  = wchar_t;
    using int_type   = wint_t;
    using off_type   = streamoff;
    using pos_type   = wstreampos;
    using state_type = mbstate_t;
+   using comparison_category = strong_ordering;
    [...]
  };
}
```

Change 21.3.1 [string.syn]:

```
namespace std {
  // [char.traits], character traits
  template<class charT> struct char_traits;
  template<> struct char_traits<char>;
  template<> struct char_traits<char8_t>;
  template<> struct char_traits<char16_t>;
  template<> struct char_traits<char32_t>;
  template<> struct char_traits<wchar_t>;

  // [basic.string], basic_string
  template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
    class basic_string;

  [...]

  template<class charT, class traits, class Allocator>
    bool operator==(const basic_string<charT, traits, Allocator>& lhs,
                    const basic_string<charT, traits, Allocator>& rhs) noexcept;
- template<class charT, class traits, class Allocator>
-   bool operator==(const charT* lhs,
-                   const basic_string<charT, traits, Allocator>& rhs);
  template<class charT, class traits, class Allocator>
    bool operator==(const basic_string<charT, traits, Allocator>& lhs,
                    const charT* rhs);
- template<class charT, class traits, class Allocator>
-   bool operator!=(const basic_string<charT, traits, Allocator>& lhs,
-                   const basic_string<charT, traits, Allocator>& rhs) noexcept;
- template<class charT, class traits, class Allocator>
-   bool operator!=(const charT* lhs,
-                   const basic_string<charT, traits, Allocator>& rhs);
- template<class charT, class traits, class Allocator>
-   bool operator!=(const basic_string<charT, traits, Allocator>& lhs,
-                   const charT* rhs);

- template<class charT, class traits, class Allocator>
-   bool operator< (const basic_string<charT, traits, Allocator>& lhs,
-                   const basic_string<charT, traits, Allocator>& rhs) noexcept;
- template<class charT, class traits, class Allocator>
-   bool operator< (const basic_string<charT, traits, Allocator>& lhs,
-                   const charT* rhs);
- template<class charT, class traits, class Allocator>
-   bool operator< (const charT* lhs,
-                   const basic_string<charT, traits, Allocator>& rhs);
- template<class charT, class traits, class Allocator>
-   bool operator> (const basic_string<charT, traits, Allocator>& lhs,
-                   const basic_string<charT, traits, Allocator>& rhs) noexcept;
- template<class charT, class traits, class Allocator>
-   bool operator> (const basic_string<charT, traits, Allocator>& lhs,
-                   const charT* rhs);
- template<class charT, class traits, class Allocator>
-   bool operator> (const charT* lhs,
-                   const basic_string<charT, traits, Allocator>& rhs);

- template<class charT, class traits, class Allocator>
-   bool operator<=(const basic_string<charT, traits, Allocator>& lhs,
-                   const basic_string<charT, traits, Allocator>& rhs) noexcept;
- template<class charT, class traits, class Allocator>
-   bool operator<=(const basic_string<charT, traits, Allocator>& lhs,
-                   const charT* rhs);
- template<class charT, class traits, class Allocator>
-   bool operator<=(const charT* lhs,
-                   const basic_string<charT, traits, Allocator>& rhs);
- template<class charT, class traits, class Allocator>
-   bool operator>=(const basic_string<charT, traits, Allocator>& lhs,
-                   const basic_string<charT, traits, Allocator>& rhs) noexcept;
- template<class charT, class traits, class Allocator>
-   bool operator>=(const basic_string<charT, traits, Allocator>& lhs,
-                   const charT* rhs);
- template<class charT, class traits, class Allocator>
-   bool operator>=(const charT* lhs,
-                   const basic_string<charT, traits, Allocator>& rhs);

+ template<class charT, class traits, class Allocator>
+   see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
+                         const basic_string<charT, traits, Allocator>& rhs) noexcept;
+ template<class charT, class traits, class Allocator>
+   see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
+                         const charT* rhs) noexcept;


  [...]
}
```

Change 21.3.3.2 [string.cmp]:

```
   template<class charT, class traits, class Allocator>
     bool operator==(const basic_string<charT, traits, Allocator>& lhs,
                     const basic_string<charT, traits, Allocator>& rhs) noexcept;
-  template<class charT, class traits, class Allocator>
-    bool operator==(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
   template<class charT, class traits, class Allocator>
     bool operator==(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

-  template<class charT, class traits, class Allocator>
-    bool operator!=(const basic_string<charT, traits, Allocator>& lhs,
-                    const basic_string<charT, traits, Allocator>& rhs) noexcept;
-  template<class charT, class traits, class Allocator>
-    bool operator!=(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
-  template<class charT, class traits, class Allocator>
-    bool operator!=(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

-  template<class charT, class traits, class Allocator>
-    bool operator< (const basic_string<charT, traits, Allocator>& lhs,
-                    const basic_string<charT, traits, Allocator>& rhs) noexcept;
-  template<class charT, class traits, class Allocator>
-    bool operator< (const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
-  template<class charT, class traits, class Allocator>
-    bool operator< (const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

-  template<class charT, class traits, class Allocator>
-    bool operator> (const basic_string<charT, traits, Allocator>& lhs,
-                    const basic_string<charT, traits, Allocator>& rhs) noexcept;
-  template<class charT, class traits, class Allocator>
-    bool operator> (const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
-  template<class charT, class traits, class Allocator>
-    bool operator> (const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

-  template<class charT, class traits, class Allocator>
-    bool operator<=(const basic_string<charT, traits, Allocator>& lhs,
                     const basic_string<charT, traits, Allocator>& rhs) noexcept;
-  template<class charT, class traits, class Allocator>
-    bool operator<=(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
-  template<class charT, class traits, class Allocator>
-    bool operator<=(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

-  template<class charT, class traits, class Allocator>
-    bool operator>=(const basic_string<charT, traits, Allocator>& lhs,
-                    const basic_string<charT, traits, Allocator>& rhs) noexcept;
-  template<class charT, class traits, class Allocator>
-    bool operator>=(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
-  template<class charT, class traits, class Allocator>
-    bool operator>=(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);

+  template<class charT, class traits, class Allocator>
+    see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
+                          const basic_string<charT, traits, Allocator>& rhs) noexcept;
+  template<class charT, class traits, class Allocator>
+    see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
+                          const charT* rhs) noexcept;
```

[1] *Effects*: Let op be the operator. Equivalent to:

```
        return basic_string_view<charT, traits>(lhs) op basic_string_view<charT, traits>(rhs);
```

Change 21.4.1 [string.view.synop]:

```
namespace std {
  // [string.view.template], class template basic_string_view
  template<class charT, class traits = char_traits<charT>>
  class basic_string_view;

  // [string.view.comparison], non-member comparison functions
  template<class charT, class traits>
    constexpr bool operator==(basic_string_view<charT, traits> x,
                              basic_string_view<charT, traits> y) noexcept;
- template<class charT, class traits>
-   constexpr bool operator!=(basic_string_view<charT, traits> x,
-                             basic_string_view<charT, traits> y) noexcept;
- template<class charT, class traits>
-   constexpr bool operator< (basic_string_view<charT, traits> x,
-                             basic_string_view<charT, traits> y) noexcept;
- template<class charT, class traits>
-   constexpr bool operator> (basic_string_view<charT, traits> x,
-                             basic_string_view<charT, traits> y) noexcept;
- template<class charT, class traits>
-   constexpr bool operator<=(basic_string_view<charT, traits> x,
```

```
-                             basic_string_view<charT, traits> y) noexcept;
- template<class charT, class traits>
-   constexpr bool operator>=(basic_string_view<charT, traits> x,
-                             basic_string_view<charT, traits> y) noexcept;
+ template<class charT, class traits>
+   constexpr see below operator<=>(basic_string_view<charT, traits> x,
+                                   basic_string_view<charT, traits> y) noexcept;

  // see [string.view.comparison], sufficient additional overloads of comparison functions

  [...]
}
```

Add `<=>` to the table at the beginning of 21.4.3 [string.view.comparisons]:

Expression	Equivalent to
`t == sv`	`S(t) == sv`
[...]	[...]
`t <=> sv`	`S(t) <=> sv`
`sv <=> t`	`sv <=> S(t)`

Change the rest of 21.4.3 [string.view.comparisons]:

```
template<class charT, class traits>
  constexpr bool operator==(basic_string_view<charT, traits> lhs,
                            basic_string_view<charT, traits> rhs) noexcept;
```

2 *Returns*: `lhs.compare(rhs) == 0`.

```
template<class charT, class traits>
  constexpr bool operator!=(basic_string_view<charT, traits> lhs,
                            basic_string_view<charT, traits> rhs) noexcept;
```

3 ~~*Returns:* `lhs.compare(rhs) != 0`.~~

```
template<class charT, class traits>
  constexpr bool operator<(basic_string_view<charT, traits> lhs,
                           basic_string_view<charT, traits> rhs) noexcept;
```

4 ~~*Returns:* `lhs.compare(rhs) < 0`.~~

```
template<class charT, class traits>
  constexpr bool operator>(basic_string_view<charT, traits> lhs,
                           basic_string_view<charT, traits> rhs) noexcept;
```

5 ~~*Returns:* `lhs.compare(rhs) >= 0`.~~

```
template<class charT, class traits>
  constexpr bool operator<=(basic_string_view<charT, traits> lhs,
                            basic_string_view<charT, traits> rhs) noexcept;
```

6 ~~*Returns:* `lhs.compare(rhs) <= 0`.~~

```
template<class charT, class traits>
  constexpr bool operator>=(basic_string_view<charT, traits> lhs,
                            basic_string_view<charT, traits> rhs) noexcept;
```

7 ~~*Returns:* `lhs.compare(rhs) >= 0`.~~

```
template<class charT, class traits>
  constexpr see below operator<=>(basic_string_view<charT, traits> lhs,
                                  basic_string_view<charT, traits> rhs) noexcept;
```

8 Let `R` denote the type `traits::comparison_category` if it exists, otherwise `R` is `weak_ordering`.

9 *Returns*: `static_cast<R>(lhs.compare(rhs) <=> 0)`.

§ 5.7 Clause 22: Containers library

`array`'s comparisons move to be hidden friends to allow for use as non-type template parameters. All the other containers drop `!=` and, if they have relational operators, those get replaced with a `<=>`.

Add to 22.2.1 [container.requirements.general]/4:

4 In Tables 62, 63, and 64 `X` denotes a container class containing objects of type `T`, `a` and `b` denote values of type `X`, `i` and `j` denote values of type (possibly-const) `X::iterator`, `u` denotes an identifier, `r` denotes a non-const value of type `X`, and `rv` denotes a non-const rvalue of type `X`.

Add a row to 22.2.1, Table 62 [container.req]:

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition	Complexity
`i <=> j`	`strong_ordering` if `X::iterator` meets the random access iterator requirements, otherwise `strong_equality`			constant

Add `<=>` to the requirements in 22.2.1 [container.requirements.general]/7:

In the expressions

```
  i == j
  i != j
  i < j
  i <= j
  i >= j
  i > j
+ i <=> j
  i - j
```

where `i` and `j` denote objects of a container's iterator type, either or both may be replaced by an object of the container's `const_iterator` type referring to the same element with no change in semantics.

Replace 22.2.1 [container.requirements.general], Table 64 [container.opt] to refer to `<=>` using *`synth-3way`* instead of `<`.

Table 64 lists operations that are provided for some types of containers but not others. Those containers for which the listed operations are provided shall implement the semantics described in Table 64 unless otherwise stated. If the iterators passed to ~~`lexicographical_compare`~~ `lexicographical_compare_three_way` satisfy the constexpr iterator requirements ([iterator.requirements.general]) then the operations described in Table 64 are implemented by constexpr functions.

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition	Complex
~~`a < b`~~	~~convertible to `bool`~~	~~`lexicographical_compare(`~~ ~~`  a.begin(), a.end(),`~~ ~~`  b.begin(), b.end())`~~	~~*Expects:* `<` is defined for values of type (possibly const) `T`. `<` is a total ordering relationship.~~	~~linear~~
~~`a > b`~~	~~convertible to `bool`~~	~~`b < a`~~		~~linear~~
~~`a <= b`~~	~~convertible to `bool`~~	~~`!(a > b)`~~		~~linear~~
~~`a >= b`~~	~~convertible to `bool`~~	~~`!(a < b)`~~		~~linear~~

`a <=> b`	*`synth-3way-result`*`<value_type>`	`lexicographical_compare_three_way(`
		`  a.begin(), a.end(),`
		`  b.begin(), b.end(),`
		`  synth-3way``)`
			Expects: Either `<=>` is defined for values of type (possibly const) `T`, or `<` is defined for values of type (possibly const) `T` and `<` is a total ordering relationship.	linear

Change 22.3.2 [array.syn]:

```
#include <initializer_list>

namespace std {
  // [array], class template array
  template<class T, size_t N> struct array;

- template<class T, size_t N>
-   constexpr bool operator==(const array<T, N>& x, const array<T, N>& y);
- template<class T, size_t N>
-   constexpr bool operator!=(const array<T, N>& x, const array<T, N>& y);
- template<class T, size_t N>
-   constexpr bool operator< (const array<T, N>& x, const array<T, N>& y);
- template<class T, size_t N>
-   constexpr bool operator> (const array<T, N>& x, const array<T, N>& y);
- template<class T, size_t N>
-   constexpr bool operator<=(const array<T, N>& x, const array<T, N>& y);
- template<class T, size_t N>
-   constexpr bool operator>=(const array<T, N>& x, const array<T, N>& y);
  template<class T, size_t N>
    constexpr void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));
  [...]
}
```

Change 22.3.3 [deque.syn]:

```
#include <initializer_list>

namespace std {
  // [deque], class template deque
  template<class T, class Allocator = allocator<T>> class deque;

  template<class T, class Allocator>
    bool operator==(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator!=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator< (const deque<T, Allocator>& x, const deque<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator> (const deque<T, Allocator>& x, const deque<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator<=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator>=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
+ template<class T, class Allocator>
+   synth-3way-result<T> operator<=>(const deque<T, Allocator>& x, const deque<T, Allocator>& y);

  [...]
}
```

Change 22.3.4 [forward.list.syn]:

```
#include <initializer_list>

namespace std {
  // [forwardlist], class template forward_list
  template<class T, class Allocator = allocator<T>> class forward_list;

  template<class T, class Allocator>
    bool operator==(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator!=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator< (const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator> (const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator<=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator>=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
+ template<class T, class Allocator>
+   synth-3way-result<T> operator<=>(const forward_list<T, Allocator>& x,
+                                    const forward_list<T, Allocator>& y);

  [...]
}
```

Change 22.3.5 [list.syn]:

```
#include <initializer_list>

namespace std {
  // [list], class template list
  template<class T, class Allocator = allocator<T>> class list;
```

```
  template<class T, class Allocator>
    bool operator==(const list<T, Allocator>& x, const list<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator!=(const list<T, Allocator>& x, const list<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator< (const list<T, Allocator>& x, const list<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator> (const list<T, Allocator>& x, const list<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator<=(const list<T, Allocator>& x, const list<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator>=(const list<T, Allocator>& x, const list<T, Allocator>& y);
+ template<class T, class Allocator>
+   synth-3way-result<T> operator<=>(const list<T, Allocator>& x, const list<T, Allocator>& y);

  [...]
}
```

Change 22.3.6 [vector.syn]:

```
#include <initializer_list>

namespace std {
  // [vector], class template vector
  template<class T, class Allocator = allocator<T>> class vector;

  template<class T, class Allocator>
    bool operator==(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator!=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator< (const vector<T, Allocator>& x, const vector<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator> (const vector<T, Allocator>& x, const vector<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator<=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
- template<class T, class Allocator>
-   bool operator>=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
+ template<class T, class Allocator>
+   synth-3way-result<T> operator<=>(const vector<T, Allocator>& x, const vector<T, Allocator>& y);

  [...]
}
```

Change 22.3.7.1 [array.overview]:

```
namespace std {
  template<class T, size_t N>
  struct array {
    [...]

    constexpr T *       data() noexcept;
    constexpr const T * data() const noexcept;

+   friend constexpr bool operator==(const array&, const array&) = default;
+   friend constexpr synth-3way-result<value_type>
+     operator<=>(const array&, const array&);
  };

  template<class T, class... U>
    array(T, U...) -> array<T, 1 + sizeof...(U)>;
}
```

Change 22.4.2 [associative.map.syn]. Instead of writing out the value type of `pair < const Key, T >`, I'm using *see above* for the return types of all the `<=>`s.

```
#include <initializer_list>

namespace std {
  // [map], class template map
  template<class Key, class T, class Compare = less<Key>,
           class Allocator = allocator<pair<const Key, T>>>
    class map;

  template<class Key, class T, class Compare, class Allocator>
    bool operator==(const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
- template<class Key, class T, class Compare, class Allocator>
-   bool operator!=(const map<Key, T, Compare, Allocator>& x,
-                   const map<Key, T, Compare, Allocator>& y);
- template<class Key, class T, class Compare, class Allocator>
```

```
-   bool operator< (const map<Key, T, Compare, Allocator>& x,
-                   const map<Key, T, Compare, Allocator>& y);
- template<class Key, class T, class Compare, class Allocator>
-   bool operator> (const map<Key, T, Compare, Allocator>& x,
-                   const map<Key, T, Compare, Allocator>& y);
- template<class Key, class T, class Compare, class Allocator>
-   bool operator<=(const map<Key, T, Compare, Allocator>& x,
-                   const map<Key, T, Compare, Allocator>& y);
- template<class Key, class T, class Compare, class Allocator>
-   bool operator>=(const map<Key, T, Compare, Allocator>& x,
-                   const map<Key, T, Compare, Allocator>& y);
+ template<class Key, class T, class Compare, class Allocator>
+   see above operator<=>(const map<Key, T, Compare, Allocator>& x,
+                         const map<Key, T, Compare, Allocator>& y);

  [...]


  // [multimap], class template multimap
  template<class Key, class T, class Compare = less<Key>,
           class Allocator = allocator<pair<const Key, T>>>
    class multimap;

  template<class Key, class T, class Compare, class Allocator>
    bool operator==(const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
- template<class Key, class T, class Compare, class Allocator>
-   bool operator!=(const multimap<Key, T, Compare, Allocator>& x,
-                   const multimap<Key, T, Compare, Allocator>& y);
- template<class Key, class T, class Compare, class Allocator>
-   bool operator< (const multimap<Key, T, Compare, Allocator>& x,
-                   const multimap<Key, T, Compare, Allocator>& y);
- template<class Key, class T, class Compare, class Allocator>
-   bool operator> (const multimap<Key, T, Compare, Allocator>& x,
-                   const multimap<Key, T, Compare, Allocator>& y);
- template<class Key, class T, class Compare, class Allocator>
-   bool operator<=(const multimap<Key, T, Compare, Allocator>& x,
-                   const multimap<Key, T, Compare, Allocator>& y);
- template<class Key, class T, class Compare, class Allocator>
-   bool operator>=(const multimap<Key, T, Compare, Allocator>& x,
-                   const multimap<Key, T, Compare, Allocator>& y);
+ template<class Key, class T, class Compare, class Allocator>
+   see above operator<=>(const multimap<Key, T, Compare, Allocator>& x,
+                         const multimap<Key, T, Compare, Allocator>& y);

  [...]
]
```

Change 22.4.3 [associative.set.syn]. These could just use `Key` directly, but decided to use *`see above`* anyway just to keep the associative containers consistent.

```
#include <initializer_list>

namespace std {
  // [set], class template set
  template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
    class set;

  template<class Key, class Compare, class Allocator>
    bool operator==(const set<Key, Compare, Allocator>& x,
                    const set<Key, Compare, Allocator>& y);
- template<class Key, class Compare, class Allocator>
-   bool operator!=(const set<Key, Compare, Allocator>& x,
-                   const set<Key, Compare, Allocator>& y);
- template<class Key, class Compare, class Allocator>
-   bool operator< (const set<Key, Compare, Allocator>& x,
-                   const set<Key, Compare, Allocator>& y);
- template<class Key, class Compare, class Allocator>
-   bool operator> (const set<Key, Compare, Allocator>& x,
-                   const set<Key, Compare, Allocator>& y);
- template<class Key, class Compare, class Allocator>
-   bool operator<=(const set<Key, Compare, Allocator>& x,
-                   const set<Key, Compare, Allocator>& y);
- template<class Key, class Compare, class Allocator>
-   bool operator>=(const set<Key, Compare, Allocator>& x,
-                   const set<Key, Compare, Allocator>& y);
+ template<class Key, class Compare, class Allocator>
+   see above operator<=>(const set<Key, Compare, Allocator>& x,
+                         const set<Key, Compare, Allocator>& y);

  [...]

  // [multiset], class template multiset
```

```
  template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
    class multiset;

  template<class Key, class Compare, class Allocator>
    bool operator==(const multiset<Key, Compare, Allocator>& x,
                    const multiset<Key, Compare, Allocator>& y);
- template<class Key, class Compare, class Allocator>
-   bool operator!=(const multiset<Key, Compare, Allocator>& x,
-                   const multiset<Key, Compare, Allocator>& y);
- template<class Key, class Compare, class Allocator>
-   bool operator< (const multiset<Key, Compare, Allocator>& x,
-                   const multiset<Key, Compare, Allocator>& y);
- template<class Key, class Compare, class Allocator>
-   bool operator> (const multiset<Key, Compare, Allocator>& x,
-                   const multiset<Key, Compare, Allocator>& y);
- template<class Key, class Compare, class Allocator>
-   bool operator<=(const multiset<Key, Compare, Allocator>& x,
-                   const multiset<Key, Compare, Allocator>& y);
- template<class Key, class Compare, class Allocator>
-   bool operator>=(const multiset<Key, Compare, Allocator>& x,
-                   const multiset<Key, Compare, Allocator>& y);
+ template<class Key, class Compare, class Allocator>
+   see above operator<=>(const multiset<Key, Compare, Allocator>& x,
+                         const multiset<Key, Compare, Allocator>& y);


  [...]
}
```

Change 22.5.2 [unord.map.syn]:

```
#include <initializer_list>

namespace std {
  // [unord.map], class template unordered_map
  template<class Key,
           class T,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Alloc = allocator<pair<const Key, T>>>
    class unordered_map;

  // [unord.multimap], class template unordered_multimap
  template<class Key,
           class T,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Alloc = allocator<pair<const Key, T>>>
    class unordered_multimap;

  template<class Key, class T, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_map<Key, T, Hash, Pred, Alloc>& a,
                    const unordered_map<Key, T, Hash, Pred, Alloc>& b);
- template<class Key, class T, class Hash, class Pred, class Alloc>
-   bool operator!=(const unordered_map<Key, T, Hash, Pred, Alloc>& a,
-                   const unordered_map<Key, T, Hash, Pred, Alloc>& b);

  template<class Key, class T, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_multimap<Key, T, Hash, Pred, Alloc>& a,
                    const unordered_multimap<Key, T, Hash, Pred, Alloc>& b);
- template<class Key, class T, class Hash, class Pred, class Alloc>
-   bool operator!=(const unordered_multimap<Key, T, Hash, Pred, Alloc>& a,
-                   const unordered_multimap<Key, T, Hash, Pred, Alloc>& b);

  [...]
}
```

Change 22.5.3 [unordered.set.syn]:

```
#include <initializer_list>

namespace std {
  // [unord.set], class template unordered_set
  template<class Key,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Alloc = allocator<Key>>
    class unordered_set;

  // [unord.multiset], class template unordered_multiset
  template<class Key,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
```

```
            class Alloc = allocator<Key>>
    class unordered_multiset;

  template<class Key, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_set<Key, Hash, Pred, Alloc>& a,
                    const unordered_set<Key, Hash, Pred, Alloc>& b);
- template<class Key, class Hash, class Pred, class Alloc>
-   bool operator!=(const unordered_set<Key, Hash, Pred, Alloc>& a,
-                   const unordered_set<Key, Hash, Pred, Alloc>& b);

  template<class Key, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_multiset<Key, Hash, Pred, Alloc>& a,
                    const unordered_multiset<Key, Hash, Pred, Alloc>& b);
- template<class Key, class Hash, class Pred, class Alloc>
-   bool operator!=(const unordered_multiset<Key, Hash, Pred, Alloc>& a,
-                   const unordered_multiset<Key, Hash, Pred, Alloc>& b);

  [...]
}
```

Change 22.6.2 [queue.syn]:

```
#include <initializer_list>

namespace std {
  template<class T, class Container = deque<T>> class queue;

  template<class T, class Container>
    bool operator==(const queue<T, Container>& x, const queue<T, Container>& y);
  template<class T, class Container>
    bool operator!=(const queue<T, Container>& x, const queue<T, Container>& y);
  template<class T, class Container>
    bool operator< (const queue<T, Container>& x, const queue<T, Container>& y);
  template<class T, class Container>
    bool operator> (const queue<T, Container>& x, const queue<T, Container>& y);
  template<class T, class Container>
    bool operator<=(const queue<T, Container>& x, const queue<T, Container>& y);
  template<class T, class Container>
    bool operator>=(const queue<T, Container>& x, const queue<T, Container>& y);
+ template<class T, ThreeWayComparable Container>
+   compare_three_way_result_t<Container>
+     operator<=>(const queue<T, Container>& x, const queue<T, Container>& y);

  [...]
}
```

Change 22.6.3 [stack.syn]:

```
#include <initializer_list>

namespace std {
  template<class T, class Container = deque<T>> class stack;

  template<class T, class Container>
    bool operator==(const stack<T, Container>& x, const stack<T, Container>& y);
  template<class T, class Container>
    bool operator!=(const stack<T, Container>& x, const stack<T, Container>& y);
  template<class T, class Container>
    bool operator< (const stack<T, Container>& x, const stack<T, Container>& y);
  template<class T, class Container>
    bool operator> (const stack<T, Container>& x, const stack<T, Container>& y);
  template<class T, class Container>
    bool operator<=(const stack<T, Container>& x, const stack<T, Container>& y);
  template<class T, class Container>
    bool operator>=(const stack<T, Container>& x, const stack<T, Container>& y);
+ template<class T, ThreeWayComparable Container>
+   compare_three_way_result_t<Container>
+     operator<=>(const stack<T, Container>& x, const stack<T, Container>& y);

  template<class T, class Container>
    void swap(stack<T, Container>& x, stack<T, Container>& y) noexcept(noexcept(x.swap(y)));
  template<class T, class Container, class Alloc>
    struct uses_allocator<stack<T, Container>, Alloc>;
}
```

Add to 22.6.4.4 [queue.ops]:

```
template<class T, class Container>
    bool operator>=(const queue<T, Container>& x,
                    const queue<T, Container>& y);
```

6 *Returns*: x .c >= y .c.

```
template<class T, ThreeWayComparable Container>
  compare_three_way_result_t<Container>
    operator<=>(const queue<T, Container>& x,
                const queue<T, Container>& y);
```

Returns: `x.c <=> y.c`.

Add to 22.6.6.4 [stack.ops]:

```
template<class T, class Container>
    bool operator>=(const stack<T, Container>& x,
                    const stack<T, Container>& y);
```

Returns: `x.c >= y.c`.

```
template<class T, ThreeWayComparable Container>
  compare_three_way_result_t<Container>
    operator<=>(const stack<T, Container>& x,
                const stack<T, Container>& y);
```

Returns: `x.c <=> y.c`.

5.8 Clause 23: Iterators library

Changing the operators for `reverse_iterator`, `move_iterator`, `istream_iterator`, `istreambuf_iterator`, `common_iterator`, `counted_iterator`, `unreachable_sentinel`.

We preserve existing comparison operators for `reverse_iterator` because `>` actually forwards to the base `>` rather than invoking the `<` with the arguments reversed. So, like `optional`, we cannot synthesize a `<=>`.

We preserve existing comparison operators `move_iterator` because it seems pretty bad to try to synthesize a three-way comparison out of two operator calls instead of just making the one operator call.

Notably, we do *not* add `<=>` to any iterator requirements, although all standard library iterators that are ordered should provide `<=>`.

Change 23.2 [iterator.synopsis]:

```
#include <concepts>

namespace std {
  [...]

  // [predef.iterators], predefined iterators and sentinels
  // [reverse.iterators], reverse iterators
  template<class Iterator> class reverse_iterator;

  template<class Iterator1, class Iterator2>
    constexpr bool operator==(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template<class Iterator1, class Iterator2>
    constexpr bool operator!=(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template<class Iterator1, class Iterator2>
    constexpr bool operator<(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template<class Iterator1, class Iterator2>
    constexpr bool operator>(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template<class Iterator1, class Iterator2>
    constexpr bool operator<=(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template<class Iterator1, class Iterator2>
    constexpr bool operator>=(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
+ template<class Iterator1, ThreeWayComparableWith<Iterator1> Iterator2>
+   constexpr compare_three_way_result_t<Iterator1, Iterator2>
+     operator<=>(const reverse_iterator<Iterator1>& x,
+                 const reverse_iterator<Iterator2>& y);

  [...]

  // [move.iterators], move iterators and sentinels
  template<class Iterator> class move_iterator;
```

```
  template<class Iterator1, class Iterator2>
    constexpr bool operator==(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
- template<class Iterator1, class Iterator2>
-   constexpr bool operator!=(
-     const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
  template<class Iterator1, class Iterator2>
    constexpr bool operator<(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
  template<class Iterator1, class Iterator2>
    constexpr bool operator>(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
  template<class Iterator1, class Iterator2>
    constexpr bool operator<=(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
  template<class Iterator1, class Iterator2>
    constexpr bool operator>=(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
+ template<class Iterator1, ThreeWayComparableWith<Iterator1> Iterator2>
+   constexpr compare_three_way_result_t<Iterator1, Iterator2>
+     operator<=>(const move_iterator<Iterator1>& x,
+                 const move_iterator<Iterator2>& y);

  [...]

  // [stream.iterators], stream iterators
  template<class T, class charT = char, class traits = char_traits<charT>,
           class Distance = ptrdiff_t>
  class istream_iterator;
  template<class T, class charT, class traits, class Distance>
    bool operator==(const istream_iterator<T,charT,traits,Distance>& x,
             const istream_iterator<T,charT,traits,Distance>& y);
- template<class T, class charT, class traits, class Distance>
-   bool operator!=(const istream_iterator<T,charT,traits,Distance>& x,
-            const istream_iterator<T,charT,traits,Distance>& y);

  template<class T, class charT = char, class traits = char_traits<charT>>
      class ostream_iterator;

  template<class charT, class traits = char_traits<charT>>
    class istreambuf_iterator;
  template<class charT, class traits>
    bool operator==(const istreambuf_iterator<charT,traits>& a,
             const istreambuf_iterator<charT,traits>& b);
- template<class charT, class traits>
-   bool operator!=(const istreambuf_iterator<charT,traits>& a,
-            const istreambuf_iterator<charT,traits>& b);

  [...]
}
```

Add `<=>` to 23.5.1.7 [reverse.iter.cmp]:

```
template<class Iterator1, class Iterator2>
  constexpr bool operator>=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
```

11 *Constraints*: `x.base()` `<=` `y.base()` is well-formed and convertible to `bool`.

12 *Returns*: `x.base()` `<=` `y.base()`.

```
template<class Iterator1, ThreeWayComparableWith<Iterator1> Iterator2>
  constexpr compare_three_way_result_t<Iterator1, Iterator2>
    operator<=>(const reverse_iterator<Iterator1>& x,
                const reverse_iterator<Iterator2>& y);
```

13 *Returns*: `x.base()` `<=>` `y.base()`.

Change 23.5.3.1 [move.iterator]:

```
namespace std {
  template<class Iterator>
  class move_iterator {
  public:
    [...]

    template<Sentinel<Iterator> S>
      friend constexpr bool
        operator==(const move_iterator& x, const move_sentinel<S>& y);
```

```
-   template<Sentinel<Iterator> S>
-     friend constexpr bool
-       operator==(const move_sentinel<S>& x, const move_iterator& y);
-   template<Sentinel<Iterator> S>
-     friend constexpr bool
-       operator!=(const move_iterator& x, const move_sentinel<S>& y);
-   template<Sentinel<Iterator> S>
-     friend constexpr bool
-       operator!=(const move_sentinel<S>& x, const move_iterator& y);
    template<SizedSentinel<Iterator> S>
      friend constexpr iter_difference_t<Iterator>
        operator-(const move_sentinel<S>& x, const move_iterator& y);

    [...]
  };
}
```

Remove `!=` and add `<=>` to 23.5.3.7 [move.iter.op.comp]:

```
template<class Iterator1, class Iterator2>
  constexpr bool operator==(const move_iterator<Iterator1>& x,
                            const move_iterator<Iterator2>& y);
template<Sentinel<Iterator> S>
  friend constexpr bool operator==(const move_iterator& x,
                                   const move_sentinel<S>& y);
```

```
template<Sentinel<Iterator> S>
  friend constexpr bool operator==(const move_sentinel<S>& x,
                                   const move_iterator& y);
```

1 *Constraints*: `x.base()` `==` `y.base()` is well-formed and convertible to `bool`.

2 *Returns*: `x.base()` `==` `y.base()`.

```
template<class Iterator1, class Iterator2>
  constexpr bool operator!=(const move_iterator<Iterator1>& x,
                            const move_iterator<Iterator2>& y);
template<Sentinel<Iterator> S>
  friend constexpr bool operator!=(const move_iterator& x,
                                   const move_sentinel<S>& y);
template<Sentinel<Iterator> S>
  friend constexpr bool operator!=(const move_sentinel<S>& x,
                                   const move_iterator& y);
```

3 *Constraints:* `x.base()` `==` `y.base()` is well-formed and convertible to `bool`.

4 *Returns:* `!(x == y)`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator<(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

5 *Constraints*: `x.base()` `<` `y.base()` is well-formed and convertible to `bool`.

6 *Returns*: `x.base()` `<` `y.base()`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator>(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

7 *Constraints*: `y.base()` `<` `x.base()` is well-formed and convertible to `bool`.

8 *Returns*: `y < x`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator<=(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

9 *Constraints*: `y.base()` `<` `x.base()` is well-formed and convertible to `bool`.

10 *Returns*: `!(y < x)`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator>=(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

11 *Constraints*: `x.base()` `<` `y.base()` is well-formed and convertible to `bool`.

12 *Returns*: `!(x < y)`.

```
template<class Iterator1, ThreeWayComparableWith<Iterator1> Iterator2>
  constexpr compare_three_way_result_t<Iterator1, Iterator2>
```

```
    operator<=>(const move_iterator<Iterator1>& x,
                const move_iterator<Iterator2>& y);
```

13 *Returns*: `x.base() <=> y.base()`.

Remove `!=` from 23.5.4.1 [common.iterator]:

```
namespace std {
  template<Iterator I, Sentinel<I> S>
    requires (!Same<I, S>)
  class common_iterator {
  public:
    [...]


    template<class I2, Sentinel<I> S2>
      requires Sentinel<S, I2>
    friend bool operator==(
      const common_iterator& x, const common_iterator<I2, S2>& y);
    template<class I2, Sentinel<I> S2>
      requires Sentinel<S, I2> && EqualityComparableWith<I, I2>
    friend bool operator==(
      const common_iterator& x, const common_iterator<I2, S2>& y);
-   template<class I2, Sentinel<I> S2>
-     requires Sentinel<S, I2>
-   friend bool operator!=(
-     const common_iterator& x, const common_iterator<I2, S2>& y);

    [...]
  };
}
```

Remove `!=` from 23.5.4.6 [common.iter.cmp]:

```
~~template<class I2, Sentinel<I> S2>~~
~~  requires Sentinel<S, I2>~~
~~friend bool operator!=(~~
~~  const common_iterator& x, const common_iterator<I2, S2>& y);~~
```

5 ~~*Effects:* Equivalent to:~~ ~~return~~ ~~!(x~~ ~~== y~~ ~~);~~

Change 23.5.6.1 [counted.iterator]:

```
namespace std {
  template<Iterator I>
  class counted_iterator {
  public:
    [...]

    template<Common<I> I2>
      friend constexpr bool operator==(
        const counted_iterator& x, const counted_iterator<I2>& y);
    friend constexpr bool operator==(
      const counted_iterator& x, default_sentinel_t);
-   friend constexpr bool operator==(
-     default_sentinel_t, const counted_iterator& x);

-   template<Common<I> I2>
-     friend constexpr bool operator!=(
-       const counted_iterator& x, const counted_iterator<I2>& y);
-   friend constexpr bool operator!=(
-     const counted_iterator& x, default_sentinel_t y);
-   friend constexpr bool operator!=(
-     default_sentinel_t x, const counted_iterator& y);

-   template<Common<I> I2>
-     friend constexpr bool operator<(
-       const counted_iterator& x, const counted_iterator<I2>& y);
-   template<Common<I> I2>
-     friend constexpr bool operator>(
-       const counted_iterator& x, const counted_iterator<I2>& y);
-   template<Common<I> I2>
-     friend constexpr bool operator<=(
-       const counted_iterator& x, const counted_iterator<I2>& y);
-   template<Common<I> I2>
-     friend constexpr bool operator>=(
-       const counted_iterator& x, const counted_iterator<I2>& y);
+   template<Common<I> I2>
+     friend constexpr strong_ordering operator<=>(
+       const counted_iterator& x, const counted_iterator<I2>& y);

    [...]
```

```
  };
}
```

Make the same changes to 23.5.6.6 [counted.iter.comp]:

```
template<Common<I> I2>
  friend constexpr bool operator==(
    const counted_iterator& x, const counted_iterator<I2>& y);
```

1 *Expects*: x and y refer to elements of the same sequence ([counted.iterator]).

2 *Effects*: Equivalent to: return x .length == y .length;

```
  friend constexpr bool operator==(
    const counted_iterator& x, default_sentinel_t);
```

~~friend constexpr bool operator==(~~
~~default_sentinel_t, const counted_iterator& x);~~

3 *Effects*: Equivalent to: return x .length == 0;

~~template<Common<I> I2>~~
~~friend constexpr bool operator!=(~~
~~const counted_iterator& x, const counted_iterator<I2>& y);~~
~~friend constexpr bool operator!=(~~
~~const counted_iterator& x, default_sentinel_t y);~~
~~friend constexpr bool operator!=(~~
~~default_sentinel_t x, const counted_iterator& y);~~

4 ~~*Effects:* Equivalent to: return !(x == y);~~

~~template<Common<I> I2>~~
~~friend constexpr bool operator<(~~
~~const counted_iterator& x, const counted_iterator<I2>& y);~~

5 ~~*Expects:* x and y refer to elements of the same sequence ([counted.iterator]).~~

6 ~~*Effects:* Equivalent to: return y .length < x .length;~~

7 ~~[*Note:* The argument order in the *Effects*: element is reversed because length counts down, not up. —*end note*]~~

~~template<Common<I> I2>~~
~~friend constexpr bool operator>(~~
~~const counted_iterator& x, const counted_iterator<I2>& y);~~

8 ~~*Effects:* Equivalent to: return y < x;~~

~~template<Common<I> I2>~~
~~friend constexpr bool operator<=(~~
~~const counted_iterator& x, const counted_iterator<I2>& y);~~

9 ~~*Effects:* Equivalent to: return !(y < x);~~

~~template<Common<I> I2>~~
~~friend constexpr bool operator>=(~~
~~const counted_iterator& x, const counted_iterator<I2>& y);~~

10 ~~*Effects:* Equivalent to: return !(x < y);~~

```
template<Common<I> I2>
  friend constexpr strong_ordering operator<=>(
    const counted_iterator& x, const counted_iterator<I2>& y);
```

11 *Expects*: x and y refer to elements of the same sequence ([counted.iterator]).

12 *Effects*: Equivalent to: return y .length <=> x .length;

13 [*Note*: The argument order in the *Effects*: element is reversed because length counts down, not up. —*end note*]

Change 23.5.7.1 [unreachable.sentinel] to just define what will become the single operator in the synopsis:

```
  namespace std {
    struct unreachable_sentinel_t {
      template<WeaklyIncrementable I>
-       friend constexpr bool operator==(unreachable_sentinel_t, const I&) noexcept;
+       friend constexpr bool operator==(unreachable_sentinel_t, const I&) noexcept
+         { return false; }
-       friend constexpr bool operator==(unreachable_sentinel_t, const I&) noexcept;
-     template<WeaklyIncrementable I>
-       friend constexpr bool operator==(const I&, unreachable_sentinel_t) noexcept;
-     template<WeaklyIncrementable I>
-       friend constexpr bool operator!=(unreachable_sentinel_t, const I&) noexcept;
```

```
-     template<WeaklyIncrementable I>
-       friend constexpr bool operator!=(const I&, unreachable_sentinel_t) noexcept;
    };
  }
```

Remove all of 23.5.7.2 [unreachable.sentinel.cmp] (which is just the definitions of `==` and `!=`)

```
~~template<WeaklyIncrementable I>~~
~~  friend constexpr bool operator==(unreachable_sentinel_t, const I&) noexcept;~~
~~template<WeaklyIncrementable I>~~
~~  friend constexpr bool operator==(const I&, unreachable_sentinel_t) noexcept;~~
```

1 ~~*Returns:* `false`.~~

```
~~template<WeaklyIncrementable I>~~
~~  friend constexpr bool operator!=(unreachable_sentinel_t, const I&) noexcept;~~
~~template<WeaklyIncrementable I>~~
~~  friend constexpr bool operator!=(const I&, unreachable_sentinel_t) noexcept;~~
```

2 ~~*Returns:* `true`.~~

Change 23.6.1 [istream.iterator]:

```
namespace std {
  template<class T, class charT = char, class traits = char_traits<charT>,
           class Distance = ptrdiff_t>
  class istream_iterator {
  public:
    [...]

    friend bool operator==(const istream_iterator& i, default_sentinel_t);
-   friend bool operator==(default_sentinel_t, const istream_iterator& i);
-   friend bool operator!=(const istream_iterator& x, default_sentinel_t y);
-   friend bool operator!=(default_sentinel_t x, const istream_iterator& y);

    [...]
  };
}
```

Change 23.6.1.2 [istream.iterator.ops]:

```
template<class T, class charT, class traits, class Distance>
  bool operator==(const istream_iterator<T,charT,traits,Distance>& x,
                  const istream_iterator<T,charT,traits,Distance>& y);
```

10 *Returns*: `x.in_stream == y.in_stream`.

```
~~friend bool operator==(default_sentinel_t, const istream_iterator& i);~~

friend bool operator==(const istream_iterator& i, default_sentinel_t);
```

11 *Returns*: `!i.in_stream`.

```
~~template<class T, class charT, class traits, class Distance>~~
~~  bool operator!=(const istream_iterator<T,charT,traits,Distance>& x,~~
~~                  const istream_iterator<T,charT,traits,Distance>& y);~~
~~friend bool operator!=(default_sentinel_t x, const istream_iterator& y);~~
~~friend bool operator!=(const istream_iterator& x, default_sentinel_t y);~~
```

12 ~~*Returns:* `!(x == y)`.~~

Change 23.6.3 [istreambuf.iterator]:

```
namespace std {
  template<class charT, class traits = char_traits<charT>>
  class istreambuf_iterator {
    [...]

-   friend bool operator==(default_sentinel_t s, const istreambuf_iterator& i);
    friend bool operator==(const istreambuf_iterator& i, default_sentinel_t s);
-   friend bool operator!=(default_sentinel_t a, const istreambuf_iterator& b);
-   friend bool operator!=(const istreambuf_iterator& a, default_sentinel_t b);

    [...]
  };
}
```

Change 23.6.3.3 [istreambuf.iterator.ops]:

```
template<class charT, class traits>
  bool operator==(const istreambuf_iterator<charT,traits>& a,
                  const istreambuf_iterator<charT,traits>& b);
```

6 *Returns*: a .equal (b).

```
friend bool operator==(default_sentinel_t s, const istreambuf_iterator& i);

friend bool operator==(const istreambuf_iterator& i, default_sentinel_t s);
```

7 *Returns*: i .equal (s).

```
template<class charT, class traits>
  bool operator!=(const istreambuf_iterator<charT,traits>& a,
                  const istreambuf_iterator<charT,traits>& b);
friend bool operator!=(default_sentinel_t a, const istreambuf_iterator& b);
friend bool operator!=(const istreambuf_iterator& a, default_sentinel_t b);
```

8 *Returns:* !a .equal (b).

§ 5.9 Clause 24: Ranges library

Remove no-longer-needed == and != operators from several iterators. Add a constrained <=> to iota_view ::iterator and transform_view ::iterator.

Change 24.6.3.3 [range.iota.iterator]:

```
namespace std::ranges {
  template<class W, class Bound>
  struct iota_view<W, Bound>::iterator {
    [...]

    friend constexpr bool operator==(const iterator& x, const iterator& y)
      requires EqualityComparable<W>;
-   friend constexpr bool operator!=(const iterator& x, const iterator& y)
-     requires EqualityComparable<W>;

    friend constexpr bool operator<(const iterator& x, const iterator& y)
      requires StrictTotallyOrdered<W>;
    friend constexpr bool operator>(const iterator& x, const iterator& y)
      requires StrictTotallyOrdered<W>;
    friend constexpr bool operator<=(const iterator& x, const iterator& y)
      requires StrictTotallyOrdered<W>;
    friend constexpr bool operator>=(const iterator& x, const iterator& y)
      requires StrictTotallyOrdered<W>;
+   friend constexpr compare_three_way_result_t<W> operator<=>(
+       const iterator& x, const iterator& y)
+     requires StrictTotallyOrdered<W> && ThreeWayComparable<W>;
    [...]
  };
}
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
  requires EqualityComparable<W>;
```

14 *Effects*: Equivalent to: return x .value_ == y .value_;

```
friend constexpr bool operator!=(const iterator& x, const iterator& y)
  requires EqualityComparable<W>;
```

15 *Effects:* Equivalent to: return !(x == y);

```
friend constexpr bool operator<(const iterator& x, const iterator& y)
  requires StrictTotallyOrdered<W>;
```

16 *Effects*: Equivalent to: return x .value_ < y .value_;

```
friend constexpr bool operator>(const iterator& x, const iterator& y)
  requires StrictTotallyOrdered<W>;
```

17 *Effects*: Equivalent to: return y < x;

```
friend constexpr bool operator<=(const iterator& x, const iterator& y)
  requires StrictTotallyOrdered<W>;
```

18 *Effects*: Equivalent to: return !(y < x);

```
friend constexpr bool operator>=(const iterator& x, const iterator& y)
  requires StrictTotallyOrdered<W>;
```

19 *Effects*: Equivalent to: return !(x < y);

```
friend constexpr compare_three_way_result_t<W>
  operator<=>(const iterator& x, const iterator& y)
    requires StrictTotallyOrdered<W> && ThreeWayComparable<W>;
```

19 *Effects*: Equivalent to: `return x.value_ <=> y.value_;`

Change 24.6.3.4 [range.iota.sentinel]:

```
namespace std::ranges {
  template<class W, class Bound>
  struct iota_view<W, Bound>::sentinel {
    [...]

    friend constexpr bool operator==(const iterator& x, const sentinel& y);
-   friend constexpr bool operator==(const sentinel& x, const iterator& y);
-   friend constexpr bool operator!=(const iterator& x, const sentinel& y);
-   friend constexpr bool operator!=(const sentinel& x, const iterator& y);
  };
}
```

```
constexpr explicit sentinel(Bound bound);
```

1 *Effects*: Initializes `bound_` with `bound`.

```
friend constexpr bool operator==(const iterator& x, const sentinel& y);
```

2 *Effects*: Equivalent to: `return x.value_ == y.bound_;`

~~`friend constexpr bool operator==(const sentinel& x, const iterator& y);`~~

3 ~~*Effects:* Equivalent to: `return y.bound_ == x;`~~

~~`friend constexpr bool operator!=(const iterator& x, const sentinel& y);`~~

4 ~~*Effects:* Equivalent to: `return !(x.bound_ == y);`~~

~~`friend constexpr bool operator!=(const sentinel& x, const iterator& y);`~~

5 ~~*Effects:* Equivalent to: `return !(y.bound_ == x);`~~

Remove `!=` from 24.7.4.3 [range.filter.iterator]:

```
namespace std::ranges {
  template<class V, class Pred>
  class filter_view<V, Pred>::iterator {
    [...]

    friend constexpr bool operator==(const iterator& x, const iterator& y)
      requires EqualityComparable<iterator_t<V>>;
-   friend constexpr bool operator!=(const iterator& x, const iterator& y)
-     requires EqualityComparable<iterator_t<V>>;

    [...]
  };
}
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
  requires EqualityComparable<iterator_t<V>>;
```

13 *Effects*: Equivalent to: `return x.current_ == y.current_;`

~~`friend constexpr bool operator!=(const iterator& x, const iterator& y)`~~
~~`  requires EqualityComparable<iterator_t<V>>;`~~

14 ~~*Effects:* Equivalent to: `return !(x.current_ == y.current_);`~~

Change 24.7.4.4 [range.filter.sentinel]:

```
namespace std::ranges {
  template<class V, class Pred>
  class filter_view<V, Pred>::sentinel {
    [...]

    friend constexpr bool operator==(const iterator& x, const sentinel& y);
-   friend constexpr bool operator==(const sentinel& x, const iterator& y);
-   friend constexpr bool operator!=(const iterator& x, const sentinel& y);
-   friend constexpr bool operator!=(const sentinel& x, const iterator& y);
  };
}
```

```
constexpr explicit sentinel(filter_view& parent);
```

1 *Effects*: Initializes `end_` with `ranges::end(parent)`.

```
constexpr sentinel_t<V> base() const;
```

Effects: Equivalent to: `return end_;`

```
friend constexpr bool operator==(const iterator& x, const sentinel& y);
```

Effects: Equivalent to: `return x.current_ == y.end_;`

```
friend constexpr bool operator==(const sentinel& x, const iterator& y);
```

Effects: Equivalent to: `return y == x;`

```
friend constexpr bool operator!=(const iterator& x, const sentinel& y);
```

Effects: Equivalent to: `return !(x == y);`

```
friend constexpr bool operator!=(const sentinel& x, const iterator& y);
```

Effects: Equivalent to: `return !(y == x);`

Change 24.7.5.3 [range.transform.iterator].

Given that the relational operators here all require `RandomAccessRange`, could we just provide a single `<=>` whose implementation is `(x - y) <=> 0`? Or at least `x <=> y` if possible, else fall back to the subtraction?

```
namespace std::ranges {
  template<class V, class F>
  template<bool Const>
  class transform_view<V, F>::iterator {
    [...]


    friend constexpr bool operator==(const iterator& x, const iterator& y)
      requires EqualityComparable<iterator_t<Base>>;
-   friend constexpr bool operator!=(const iterator& x, const iterator& y)
-     requires EqualityComparable<iterator_t<Base>>;

    friend constexpr bool operator<(const iterator& x, const iterator& y)
      requires RandomAccessRange<Base>;
    friend constexpr bool operator>(const iterator& x, const iterator& y)
      requires RandomAccessRange<Base>;
    friend constexpr bool operator<=(const iterator& x, const iterator& y)
      requires RandomAccessRange<Base>;
    friend constexpr bool operator>=(const iterator& x, const iterator& y)
      requires RandomAccessRange<Base>;
+   friend constexpr compare_three_way_result_t<iterator_t<Base>>
+     operator<=>(const iterator& x, const iterator& y)
+       requires RandomAccessRange<Base> && ThreeWayComparable<iterator_t<Base>>;

    [...]
  };
}
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
  requires EqualityComparable<iterator_t<Base>>;
```

Effects: Equivalent to: `return x.current_ == y.current_;`

```
friend constexpr bool operator!=(const iterator& x, const iterator& y)
  requires EqualityComparable<iterator_t<Base>>;
```

Effects: Equivalent to: `return !(x == y);`

```
friend constexpr bool operator<(const iterator& x, const iterator& y)
  requires RandomAccessRange<Base>;
```

Effects: Equivalent to: `return x.current_ < y.current_;`

```
friend constexpr bool operator>(const iterator& x, const iterator& y)
  requires RandomAccessRange<Base>;
```

Effects: Equivalent to: `return y < x;`

```
friend constexpr bool operator<=(const iterator& x, const iterator& y)
  requires RandomAccessRange<Base>;
```

Effects: Equivalent to: `return !(y < x);`

```
friend constexpr bool operator>=(const iterator& x, const iterator& y)
  requires RandomAccessRange<Base>;
```

Effects: Equivalent to: `return !(x < y);`

```
        friend constexpr compare_three_way_result_t<iterator_t<Base>>
          operator<=>(const iterator& x, const iterator& y)
            requires RandomAccessRange<Base> && ThreeWayComparable<iterator_t<Base>>;
```

19 *Effects*: Equivalent to `return x.current_ <=> y.current_;`

Change 24.7.5.4 [range.transform.sentinel]:

```
        namespace std::ranges {
          template<class V, class F>
          template<bool Const>
          class transform_view<V, F>::sentinel {
            [...]

            friend constexpr bool operator==(const iterator<Const>& x, const sentinel& y);
-           friend constexpr bool operator==(const sentinel& x, const iterator<Const>& y);
-           friend constexpr bool operator!=(const iterator<Const>& x, const sentinel& y);
-           friend constexpr bool operator!=(const sentinel& x, const iterator<Const>& y);

            [...]
          };
        }

        friend constexpr bool operator==(const iterator<Const>& x, const sentinel& y);
```

4 *Effects*: Equivalent to: `return x.current_ == y.end_;`

~~`friend constexpr bool operator==(const sentinel& x, const iterator<Const>& y);`~~

5 ~~*Effects:* Equivalent to: `return y == x;`~~

~~`friend constexpr bool operator!=(const iterator<Const>& x, const sentinel& y);`~~

6 ~~*Effects:* Equivalent to: `return !(x == y);`~~

~~`friend constexpr bool operator!=(const sentinel& x, const iterator<Const>& y);`~~

7 ~~*Effects:* Equivalent to: `return !(y == x);`~~

Change 24.7.6.3 [range.take.sentinel]:

```
        namespace std::ranges {
          template<class V>
          template<bool Const>
          class take_view<V>::sentinel {
            [...]

-           friend constexpr bool operator==(const sentinel& x, const CI& y);
            friend constexpr bool operator==(const CI& y, const sentinel& x);
-           friend constexpr bool operator!=(const sentinel& x, const CI& y);
-           friend constexpr bool operator!=(const CI& y, const sentinel& x);
          };
        }
```

~~`friend constexpr bool operator==(const sentinel& x, const CI& y);`~~

```
        friend constexpr bool operator==(const CI& y, const sentinel& x);
```

4 *Effects*: Equivalent to: `return y.count() == 0 || y.base() == x.end_;`

~~`friend constexpr bool operator!=(const sentinel& x, const CI& y);`~~
~~`friend constexpr bool operator!=(const CI& y, const sentinel& x);`~~

5 ~~*Effects:* Equivalent to: `return !(x == y);`~~

Change 24.7.7.3 [range.join.iterator]:

```
        namespace std::ranges {
        template<class V>
          template<bool Const>
          struct join_view<V>::iterator {
            [...]

            friend constexpr bool operator==(const iterator& x, const iterator& y)
              requires ref_is_glvalue && EqualityComparable<iterator_t<Base>> &&
                       EqualityComparable<iterator_t<iter_reference_t<iterator_t<Base>>>>;

-           friend constexpr bool operator!=(const iterator& x, const iterator& y)
-             requires ref_is_glvalue && EqualityComparable<iterator_t<Base>> &&
-                      EqualityComparable<iterator_t<iter_reference_t<iterator_t<Base>>>>;
```

```
    [...]
  };
}
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
  requires ref_is_glvalue && EqualityComparable<iterator_t<Base>> &&
           EqualityComparable<iterator_t<iter_reference_t<iterator_t<Base>>>>;
```

16 *Effects*: Equivalent to: `return x.outer_ == y.outer_ && x.inner_ == y.inner_;`

```
~~friend constexpr bool operator!=(const iterator& x, const iterator& y)~~
~~  requires ref_is_glvalue && EqualityComparable<iterator_t<Base>> &&~~
~~           EqualityComparable<iterator_t<iter_reference_t<iterator_t<Base>>>>;~~
```

17 ~~*Effects:* Equivalent to: `return !(x == y);`~~

Change 24.7.7.4 [range.join.sentinel]:

```
namespace std::ranges {
  template<class V>
  template<bool Const>
  struct join_view<V>::sentinel {
    [...]

    friend constexpr bool operator==(const iterator<Const>& x, const sentinel& y);
-   friend constexpr bool operator==(const sentinel& x, const iterator<Const>& y);
-   friend constexpr bool operator!=(const iterator<Const>& x, const sentinel& y);
-   friend constexpr bool operator!=(const sentinel& x, const iterator<Const>& y);
  };
}
```

```
friend constexpr bool operator==(const iterator<Const>& x, const sentinel& y);
```

3 *Effects*: Equivalent to: `return x.outer_ == y.end_;`

```
~~friend constexpr bool operator==(const sentinel& x, const iterator<Const>& y);~~
```

4 ~~*Effects:* Equivalent to: `return y == x;`~~

```
~~friend constexpr bool operator!=(const iterator<Const>& x, const sentinel& y);~~
```

5 ~~*Effects:* Equivalent to: `return !(x == y);`~~

```
~~friend constexpr bool operator!=(const sentinel& x, const iterator<Const>& y);~~
```

6 ~~*Effects:* Equivalent to: `return !(y == x);`~~

Change 24.7.8.3 [range.split.outer]:

```
namespace std::ranges {
  template<class V, class Pattern>
  template<bool Const>
  struct split_view<V, Pattern>::outer_iterator {
    [...]

    friend constexpr bool operator==(const outer_iterator& x, const outer_iterator& y)
      requires ForwardRange<Base>;
-   friend constexpr bool operator!=(const outer_iterator& x, const outer_iterator& y)
-     requires ForwardRange<Base>;

    friend constexpr bool operator==(const outer_iterator& x, default_sentinel_t);
-   friend constexpr bool operator==(default_sentinel_t, const outer_iterator& x);
-   friend constexpr bool operator!=(const outer_iterator& x, default_sentinel_t y);
-   friend constexpr bool operator!=(default_sentinel_t y, const outer_iterator& x);
  };
}
```

```
friend constexpr bool operator==(const outer_iterator& x, const outer_iterator& y)
  requires ForwardRange<Base>;
```

7 *Effects*: Equivalent to: `return x.current_ == y.current_;`

```
~~friend constexpr bool operator!=(const outer_iterator& x, const outer_iterator& y)~~
~~  requires ForwardRange<Base>;~~
```

8 ~~*Effects:* Equivalent to: `return !(x == y);`~~

```
friend constexpr bool operator==(const outer_iterator& x, default_sentinel_t);
```

```
~~friend constexpr bool operator==(default_sentinel_t, const outer_iterator& x);~~
```

9 *Effects*: Equivalent to: `return x.current == ranges::end(x.parent_->base_);`

```
~~friend constexpr bool operator!=(const outer_iterator& x, default_sentinel_t y);~~
~~friend constexpr bool operator!=(default_sentinel_t y, const outer_iterator& x);~~
```

10 ~~*Effects:* Equivalent to: `return !(x == y);`~~

Change 24.7.8.5 [range.split.inner]:

```
namespace std::ranges {
  template<class V, class Pattern>
  template<bool Const>
  struct split_view<V, Pattern>::inner_iterator {
    [...]

    friend constexpr bool operator==(const inner_iterator& x, const inner_iterator& y)
      requires ForwardRange<Base>;
-   friend constexpr bool operator!=(const inner_iterator& x, const inner_iterator& y)
-     requires ForwardRange<Base>;

    friend constexpr bool operator==(const inner_iterator& x, default_sentinel_t);
-   friend constexpr bool operator==(default_sentinel_t, const inner_iterator& x);
-   friend constexpr bool operator!=(const inner_iterator& x, default_sentinel_t y);
-   friend constexpr bool operator!=(default_sentinel_t y, const inner_iterator& x);

    [...]
  };
}
```

```
friend constexpr bool operator==(const inner_iterator& x, const inner_iterator& y)
  requires ForwardRange<Base>;
```

4 *Effects*: Equivalent to: `return x.i_.current_ == y.i_.current_;`

```
~~friend constexpr bool operator!=(const inner_iterator& x, const inner_iterator& y)~~
~~  requires ForwardRange<Base>;~~
```

5 ~~*Effects:* Equivalent to: `return !(x == y);`~~

```
friend constexpr bool operator==(const inner_iterator& x, default_sentinel_t);
```

```
~~friend constexpr bool operator==(default_sentinel_t, const inner_iterator& x);~~
```

6 *Effects*: Equivalent to:

```
auto cur = x.i_.current;
auto end = ranges::end(x.i_.parent_->base_);
if (cur == end) return true;
auto [pcur, pend] = subrange{x.i_.parent_->pattern_};
if (pcur == pend) return x.incremented_;
do {
  if (*cur != *pcur) return false;
  if (++pcur == pend) return true;
} while (++cur != end);
return false;
```

```
~~friend constexpr bool operator!=(const inner_iterator& x, default_sentinel_t y);~~
~~friend constexpr bool operator!=(default_sentinel_t y, const inner_iterator& x);~~
```

7 ~~*Effects:* Equivalent to: `return !(x == y);`~~

§ 5.10 Clause 25: Algorithms library

Remove `compare_3way` and rename `lexicographical_compare_3way`.

Change 25.4 [algorithm.syn]:

```
namespace std {
   [...]

  // [alg.3way], three-way comparison algorithms
- template<class T, class U>
-   constexpr auto compare_3way(const T& a, const U& b);
  template<class InputIterator1, class InputIterator2, class Cmp>
    constexpr auto
-     lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
+     lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                      InputIterator2 b2, InputIterator2 e2,
                                      Cmp comp)
        -> common_comparison_category_t<decltype(comp(*b1, *b2)), strong_ordering>;
```

```
  template<class InputIterator1, class InputIterator2>
    constexpr auto
-     lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
+     lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                    InputIterator2 b2, InputIterator2 e2);

   [...]
}
```

Change 25.7.11 [alg.3way]:

```
template<class T, class U> constexpr auto compare_3way(const T& a, const U& b);
```

1 *Effects:* Compares two values and produces a result of the strongest applicable comparison category type:

(1.1) — Returns `a <=> b` if that expression is well-formed.

(1.2) — Otherwise, if the expressions `a == b` and `a < b` are each well-formed and convertible to `bool`, returns `strong_ordering::equal` when `a == b` is `true`, otherwise returns `strong_ordering::less` when `a < b` is `true`, and otherwise returns `strong_ordering::greater`.

(1.3) — Otherwise, if the expression `a == b` is well-formed and convertible to `bool`, returns `strong_equality::equal` when `a == b` is `true`, and otherwise returns `strong_equality::nonequal`.

(1.4) — Otherwise, the function is defined as deleted.

```
  template<class InputIterator1, class InputIterator2, class Cmp>
    constexpr auto
-     lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
+     lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                    InputIterator2 b2, InputIterator2 e2,
                                    Cmp comp)
      -> common_comparison_category_t<decltype(comp(*b1, *b2)), strong_ordering>;
```

2 *Requires*: `Cmp` shall be a function object type whose return type is a comparison category type. *Effects*: Lexicographically compares two ranges and produces a result of the strongest applicable comparison category type. Equivalent to:

```
for ( ; b1 != e1 && b2 != e2; void(++b1), void(++b2) )
  if (auto cmp = comp(*b1,*b2); cmp != 0)
    return cmp;
return b1 != e1 ? strong_ordering::greater :
       b2 != e2 ? strong_ordering::less :
                  strong_ordering::equal;
```

```
  template<class InputIterator1, class InputIterator2>
    constexpr auto
-     lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
+     lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
                                    InputIterator2 b2, InputIterator2 e2);
```

4 *Effects*: Equivalent to:

```
  return lexicographical_compare_3way(b1, e1, b2, e2,
-                                     [](const auto& t, const auto& u) {
-                                       return compare_3way(t, u);
-                                     });
+                                     compare_three_way());
```

§ 5.11 Clause 26: Numerics library

Remove obsolete `==` and `!=` operators from `complex`, add a new `==` to `slice`.

Change 26.4.1 [complex.syn]:

```
namespace std {
  // [complex], class template complex
  template<class T> class complex;

  // [complex.special], specializations
  template<> class complex<float>;
  template<> class complex<double>;
  template<> class complex<long double>;

  [...]

  template<class T> constexpr bool operator==(const complex<T>&, const complex<T>&);
  template<class T> constexpr bool operator==(const complex<T>&, const T&);
- template<class T> constexpr bool operator==(const T&, const complex<T>&);
```

```
- template<class T> constexpr bool operator!=(const complex<T>&, const complex<T>&);
- template<class T> constexpr bool operator!=(const complex<T>&, const T&);
- template<class T> constexpr bool operator!=(const T&, const complex<T>&);

  [...]
}
```

Change 26.4.6 [complex.ops]:

```
template<class T> constexpr bool operator==(const complex<T>& lhs, const complex<T>& rhs);
template<class T> constexpr bool operator==(const complex<T>& lhs, const T& rhs);

~~template<class T> constexpr bool operator==(const T& lhs, const complex<T>& rhs);~~
```

9 *Returns*: `lhs.real() == rhs.real() && lhs.imag() == rhs.imag()`.

10 *Remarks*: The imaginary part is assumed to be `T()`, or `0.0`, for the `T` arguments.

```
~~template<class T> constexpr bool operator!=(const complex<T>& lhs, const complex<T>& rhs);~~
~~template<class T> constexpr bool operator!=(const complex<T>& lhs, const T& rhs);~~
~~template<class T> constexpr bool operator!=(const T& lhs, const complex<T>& rhs);~~
```

11 ~~Returns: `rhs.real() != lhs.real() || rhs.imag() != lhs.imag()`.~~

Change 26.7.4.1 [class.slice.overview]:

```
namespace std {
  class slice {
  public:
    slice();
    slice(size_t, size_t, size_t);

    size_t start() const;
    size_t size() const;
    size_t stride() const;

+   friend bool operator==(const slice& x, const slice& y);
  };
}
```

Add a new subclause 26.7.4.4 “Operators” [slice.ops]:

```
friend bool operator==(const slice& x, const slice& y);
```

1 *Effects*: Equivalent to:

```
return x.start() == y.start() &&
  x.size() == y.size() &&
  x.stride() == y.stride();
```

§ 5.12 Clause 27: Time library

Add `<=>` to all the chrono types where possible, in some cases added as a new constrained function template, in most cases replacing the relational operators. Also removing no-longer-necessary `!=` operators.

Change 27.2 [time.syn]:

```
namespace std {
  namespace chrono {
    [...]

    // [time.duration.comparisons], duration comparisons
    template<class Rep1, class Period1, class Rep2, class Period2>
      constexpr bool operator==(const duration<Rep1, Period1>& lhs,
                                const duration<Rep2, Period2>& rhs);
-   template<class Rep1, class Period1, class Rep2, class Period2>
-     constexpr bool operator!=(const duration<Rep1, Period1>& lhs,
-                               const duration<Rep2, Period2>& rhs);
    template<class Rep1, class Period1, class Rep2, class Period2>
      constexpr bool operator< (const duration<Rep1, Period1>& lhs,
                                const duration<Rep2, Period2>& rhs);
    template<class Rep1, class Period1, class Rep2, class Period2>
      constexpr bool operator> (const duration<Rep1, Period1>& lhs,
                                const duration<Rep2, Period2>& rhs);
    template<class Rep1, class Period1, class Rep2, class Period2>
      constexpr bool operator<=(const duration<Rep1, Period1>& lhs,
                                const duration<Rep2, Period2>& rhs);
    template<class Rep1, class Period1, class Rep2, class Period2>
      constexpr bool operator>=(const duration<Rep1, Period1>& lhs,
```

```
                                const duration<Rep2, Period2>& rhs);
+   template<class Rep1, class Period1, class Rep2, class Period2>
+       requires see below
+     constexpr auto operator<=>(const duration<Rep1, Period1>& lhs,
+                                const duration<Rep2, Period2>& rhs);

    [...]

    // [time.point.comparisons], time_point comparisons
    template<class Clock, class Duration1, class Duration2>
       constexpr bool operator==(const time_point<Clock, Duration1>& lhs,
                                 const time_point<Clock, Duration2>& rhs);
-   template<class Clock, class Duration1, class Duration2>
-      constexpr bool operator!=(const time_point<Clock, Duration1>& lhs,
-                                const time_point<Clock, Duration2>& rhs);
    template<class Clock, class Duration1, class Duration2>
       constexpr bool operator< (const time_point<Clock, Duration1>& lhs,
                                 const time_point<Clock, Duration2>& rhs);
    template<class Clock, class Duration1, class Duration2>
       constexpr bool operator> (const time_point<Clock, Duration1>& lhs,
                                 const time_point<Clock, Duration2>& rhs);
    template<class Clock, class Duration1, class Duration2>
       constexpr bool operator<=(const time_point<Clock, Duration1>& lhs,
                                 const time_point<Clock, Duration2>& rhs);
    template<class Clock, class Duration1, class Duration2>
       constexpr bool operator>=(const time_point<Clock, Duration1>& lhs,
                                 const time_point<Clock, Duration2>& rhs);
+   template<class Clock, class Duration1, ThreeWayComparableWith<Duration1> Duration2>
+      constexpr auto operator<=>(const time_point<Clock, Duration1>& lhs,
+                                 const time_point<Clock, Duration2>& rhs);

    [...]

    // [time.cal.day], class day
    class day;

    constexpr bool operator==(const day& x, const day& y) noexcept;
-   constexpr bool operator!=(const day& x, const day& y) noexcept;
-   constexpr bool operator< (const day& x, const day& y) noexcept;
-   constexpr bool operator> (const day& x, const day& y) noexcept;
-   constexpr bool operator<=(const day& x, const day& y) noexcept;
-   constexpr bool operator>=(const day& x, const day& y) noexcept;
+   constexpr strong_ordering operator<=>(const day& x, const day& y) noexcept;

    [...]

    // [time.cal.month], class month
    class month;

    constexpr bool operator==(const month& x, const month& y) noexcept;
-   constexpr bool operator!=(const month& x, const month& y) noexcept;
-   constexpr bool operator< (const month& x, const month& y) noexcept;
-   constexpr bool operator> (const month& x, const month& y) noexcept;
-   constexpr bool operator<=(const month& x, const month& y) noexcept;
-   constexpr bool operator>=(const month& x, const month& y) noexcept;
+   constexpr strong_ordering operator<=>(const month& x, const month& y) noexcept;

    [...]


    // [time.cal.year], class year
    class year;

    constexpr bool operator==(const year& x, const year& y) noexcept;
-   constexpr bool operator!=(const year& x, const year& y) noexcept;
-   constexpr bool operator< (const year& x, const year& y) noexcept;
-   constexpr bool operator> (const year& x, const year& y) noexcept;
-   constexpr bool operator<=(const year& x, const year& y) noexcept;
-   constexpr bool operator>=(const year& x, const year& y) noexcept;
+   constexpr strong_ordering operator<=>(const year& x, const year& y) noexcept;

    [...]

    // [time.cal.wd], class weekday
    class weekday;

    constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
-   constexpr bool operator!=(const weekday& x, const weekday& y) noexcept;

    [...]

    // [time.cal.wdidx], class weekday_indexed
    class weekday_indexed;
```

```
    constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
-   constexpr bool operator!=(const weekday_indexed& x, const weekday_indexed& y) noexcept;

    [...]

    // [time.cal.wdlast], class weekday_last
    class weekday_last;

    constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
-   constexpr bool operator!=(const weekday_last& x, const weekday_last& y) noexcept;

    [...]

    // [time.cal.md], class month_day
    class month_day;

    constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
-   constexpr bool operator!=(const month_day& x, const month_day& y) noexcept;
-   constexpr bool operator< (const month_day& x, const month_day& y) noexcept;
-   constexpr bool operator> (const month_day& x, const month_day& y) noexcept;
-   constexpr bool operator<=(const month_day& x, const month_day& y) noexcept;
-   constexpr bool operator>=(const month_day& x, const month_day& y) noexcept;
+   constexpr strong_ordering operator<=>(const month_day& x, const month_day& y) noexcept;

    [...]


    // [time.cal.mdlast], class month_day_last
    class month_day_last;

    constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
-   constexpr bool operator!=(const month_day_last& x, const month_day_last& y) noexcept;
-   constexpr bool operator< (const month_day_last& x, const month_day_last& y) noexcept;
-   constexpr bool operator> (const month_day_last& x, const month_day_last& y) noexcept;
-   constexpr bool operator<=(const month_day_last& x, const month_day_last& y) noexcept;
-   constexpr bool operator>=(const month_day_last& x, const month_day_last& y) noexcept;
+   constexpr strong_ordering operator<=>(const month_day_last& x, const month_day_last& y) noexcept;

    template<class charT, class traits>
      basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const month_day_last& mdl);

    // [time.cal.mwd], class month_weekday
    class month_weekday;

    constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
-   constexpr bool operator!=(const month_weekday& x, const month_weekday& y) noexcept;

    template<class charT, class traits>
      basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const month_weekday& mwd);

    // [time.cal.mwdlast], class month_weekday_last
    class month_weekday_last;

    constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
-   constexpr bool operator!=(const month_weekday_last& x, const month_weekday_last& y) noexcept;

    template<class charT, class traits>
      basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const month_weekday_last& mwdl);

    // [time.cal.ym], class year_month
    class year_month;

    constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
-   constexpr bool operator!=(const year_month& x, const year_month& y) noexcept;
-   constexpr bool operator< (const year_month& x, const year_month& y) noexcept;
-   constexpr bool operator> (const year_month& x, const year_month& y) noexcept;
-   constexpr bool operator<=(const year_month& x, const year_month& y) noexcept;
-   constexpr bool operator>=(const year_month& x, const year_month& y) noexcept;
+   constexpr strong_ordering operator<=>(const year_month& x, const year_month& y) noexcept;

    [...]

    // [time.cal.ymd], class year_month_day
    class year_month_day;

    constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
-   constexpr bool operator!=(const year_month_day& x, const year_month_day& y) noexcept;
-   constexpr bool operator< (const year_month_day& x, const year_month_day& y) noexcept;
-   constexpr bool operator> (const year_month_day& x, const year_month_day& y) noexcept;
```

```
-   constexpr bool operator<=(const year_month_day& x, const year_month_day& y) noexcept;
-   constexpr bool operator>=(const year_month_day& x, const year_month_day& y) noexcept;
+   constexpr strong_ordering operator<=>(const year_month_day& x, const year_month_day& y) noexcept;

    [...]

    // [time.cal.ymdlast], class year_month_day_last
    class year_month_day_last;

    constexpr bool operator==(const year_month_day_last& x,
                              const year_month_day_last& y) noexcept;
-   constexpr bool operator!=(const year_month_day_last& x,
-                             const year_month_day_last& y) noexcept;
-   constexpr bool operator< (const year_month_day_last& x,
-                             const year_month_day_last& y) noexcept;
-   constexpr bool operator> (const year_month_day_last& x,
-                             const year_month_day_last& y) noexcept;
-   constexpr bool operator<=(const year_month_day_last& x,
-                             const year_month_day_last& y) noexcept;
-   constexpr bool operator>=(const year_month_day_last& x,
-                             const year_month_day_last& y) noexcept;
+   constexpr strong_ordering operator<=>(const year_month_day_last& x,
+                                         const year_month_day_last& y) noexcept;

    [...]


    // [time.cal.ymwd], class year_month_weekday
    class year_month_weekday;

    constexpr bool operator==(const year_month_weekday& x,
                              const year_month_weekday& y) noexcept;
-   constexpr bool operator!=(const year_month_weekday& x,
-                             const year_month_weekday& y) noexcept;

    [...]

    // [time.cal.ymwdlast], class year_month_weekday_last
    class year_month_weekday_last;

    constexpr bool operator==(const year_month_weekday_last& x,
                              const year_month_weekday_last& y) noexcept;
-   constexpr bool operator!=(const year_month_weekday_last& x,
-                             const year_month_weekday_last& y) noexcept;

    [...]

    // [time.zone.timezone], class time_zone
    enum class choose {earliest, latest};
    class time_zone;

    bool operator==(const time_zone& x, const time_zone& y) noexcept;
-   bool operator!=(const time_zone& x, const time_zone& y) noexcept;

-   bool operator<(const time_zone& x, const time_zone& y) noexcept;
-   bool operator>(const time_zone& x, const time_zone& y) noexcept;
-   bool operator<=(const time_zone& x, const time_zone& y) noexcept;
-   bool operator>=(const time_zone& x, const time_zone& y) noexcept;
+   strong_ordering operator<=>(const time_zone& x, const time_zone& y) noexcept;

    // [time.zone.zonedtraits], class template zoned_traits
    template<class T> struct zoned_traits;

    // [time.zone.zonedtime], class template zoned_time
    template<class Duration, class TimeZonePtr = const time_zone*> class zoned_time;

    using zoned_seconds = zoned_time<seconds>;

    template<class Duration1, class Duration2, class TimeZonePtr>
      bool operator==(const zoned_time<Duration1, TimeZonePtr>& x,
                      const zoned_time<Duration2, TimeZonePtr>& y);

-   template<class Duration1, class Duration2, class TimeZonePtr>
-     bool operator!=(const zoned_time<Duration1, TimeZonePtr>& x,
-                     const zoned_time<Duration2, TimeZonePtr>& y);

    [...]

    // [time.zone.leap], leap second support
    class leap;

    bool operator==(const leap& x, const leap& y);
-   bool operator!=(const leap& x, const leap& y);
```

```
-   bool operator< (const leap& x, const leap& y);
-   bool operator> (const leap& x, const leap& y);
-   bool operator<=(const leap& x, const leap& y);
-   bool operator>=(const leap& x, const leap& y);
+   strong_ordering operator<=>(const leap& x, const leap& y);

    template<class Duration>
      bool operator==(const leap& x, const sys_time<Duration>& y);
-   template<class Duration>
-     bool operator==(const sys_time<Duration>& x, const leap& y);
-   template<class Duration>
-     bool operator!=(const leap& x, const sys_time<Duration>& y);
-   template<class Duration>
-     bool operator!=(const sys_time<Duration>& x, const leap& y);
-   template<class Duration>
-     bool operator< (const leap& x, const sys_time<Duration>& y);
-   template<class Duration>
-     bool operator< (const sys_time<Duration>& x, const leap& y);
-   template<class Duration>
-     bool operator> (const leap& x, const sys_time<Duration>& y);
-   template<class Duration>
-     bool operator> (const sys_time<Duration>& x, const leap& y);
-   template<class Duration>
-     bool operator<=(const leap& x, const sys_time<Duration>& y);
-   template<class Duration>
-     bool operator<=(const sys_time<Duration>& x, const leap& y);
-   template<class Duration>
-     bool operator>=(const leap& x, const sys_time<Duration>& y);
-   template<class Duration>
-     bool operator>=(const sys_time<Duration>& x, const leap& y);
+   template<class Duration>
+     auto operator<=>(const leap& x, const sys_time<Duration>& y);


    // [time.zone.link], class link
    class link;

    bool operator==(const link& x, const link& y);
-   bool operator!=(const link& x, const link& y);
-   bool operator< (const link& x, const link& y);
-   bool operator> (const link& x, const link& y);
-   bool operator<=(const link& x, const link& y);
-   bool operator>=(const link& x, const link& y);
+   strong_ordering operator<=>(const link& x, const link& y);

    [...]
  }
}
```

Change 27.5.6 [time.duration.comparisons]:

1 In the function descriptions that follow, `CT` represents `common_type_t <A, B>`, where `A` and `B` are the types of the two arguments to the function.

```
template<class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator==(const duration<Rep1, Period1>& lhs,
                            const duration<Rep2, Period2>& rhs);
```

2 *Returns*: `CT(lhs).count() == CT(rhs).count()`.

```
~~template<class Rep1, class Period1, class Rep2, class Period2>~~
~~  constexpr bool operator!=(const duration<Rep1, Period1>& lhs,~~
~~                            const duration<Rep2, Period2>& rhs);~~
```

3 ~~*Returns:* `!(lhs == rhs)`.~~

```
template<class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator<(const duration<Rep1, Period1>& lhs,
                           const duration<Rep2, Period2>& rhs);
```

4 *Returns*: `CT(lhs).count() < CT(rhs).count()`.

```
template<class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator>(const duration<Rep1, Period1>& lhs,
                           const duration<Rep2, Period2>& rhs);
```

5 *Returns*: `rhs < lhs`.

```
template<class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator<=(const duration<Rep1, Period1>& lhs,
                            const duration<Rep2, Period2>& rhs);
```

Returns: `!(rhs < lhs).`

```
template<class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator>=(const duration<Rep1, Period1>& lhs,
                            const duration<Rep2, Period2>& rhs);
```

Returns: `!(lhs < rhs).`

```
template<class Rep1, class Period1, class Rep2, class Period2>
    requires ThreeWayComparable<typename CT::rep>
  constexpr auto operator<=>(const duration<Rep1, Period1>& lhs,
                             const duration<Rep2, Period2>& rhs);
```

Returns: `CT(lhs).count() <=> CT(rhs).count()`.

Change 27.6.6 [time.point.comparisons]:

```
template<class Clock, class Duration1, class Duration2>
  constexpr bool operator==(const time_point<Clock, Duration1>& lhs,
                            const time_point<Clock, Duration2>& rhs);
```

Returns: `lhs.time_since_epoch() == rhs.time_since_epoch()`.

~~`template<class Clock, class Duration1, class Duration2>`~~
~~`  constexpr bool operator!=(const time_point<Clock, Duration1>& lhs,`~~
~~`                            const time_point<Clock, Duration2>& rhs);`~~

~~*Returns:* `!(lhs == rhs)`.~~

```
template<class Clock, class Duration1, class Duration2>
  constexpr bool operator<(const time_point<Clock, Duration1>& lhs,
                           const time_point<Clock, Duration2>& rhs);
```

Returns: `lhs.time_since_epoch() < rhs.time_since_epoch()`.

```
template<class Clock, class Duration1, class Duration2>
  constexpr bool operator>(const time_point<Clock, Duration1>& lhs,
                           const time_point<Clock, Duration2>& rhs);
```

Returns: `rhs < lhs`.

```
template<class Clock, class Duration1, class Duration2>
  constexpr bool operator<=(const time_point<Clock, Duration1>& lhs,
                            const time_point<Clock, Duration2>& rhs);
```

Returns: `!(rhs < lhs)`.

```
template<class Clock, class Duration1, class Duration2>
  constexpr bool operator>=(const time_point<Clock, Duration1>& lhs,
                            const time_point<Clock, Duration2>& rhs);
```

Returns: `!(lhs < rhs)`.

```
template<class Clock, class Duration1,
         ThreeWayComparableWith<Duration1> Duration2>
  constexpr auto operator<=>(const time_point<Clock, Duration1>& lhs,
                             const time_point<Clock, Duration2>& rhs);
```

Returns: `lhs.time_since_epoch() <=> rhs.time_since_epoch()`.

Change 27.8.3.3 [time.cal.day.nonmembers]:

```
constexpr bool operator==(const day& x, const day& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.

~~`constexpr bool operator<(const day& x, const day& y) noexcept;`~~

~~*Returns:* `unsigned{x} <= unsigned{y}`.~~

```
constexpr strong_ordering operator<=>(const day& x, const day& y) noexcept;
```

Returns: `unsigned{x} <=> unsigned{y}`.

Change 27.8.4.3 [time.cal.month.nonmembers]:

```
constexpr bool operator==(const month& x, const month& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.

~~`constexpr bool operator<(const month& x, const month& y) noexcept;`~~

~~*Returns:* `unsigned {x } <= unsigned {y };`~~

```
constexpr strong_ordering operator<=>(const month& x, const month& y) noexcept;
```

Returns: `unsigned {x } <=> unsigned {y }`.

Change 27.8.5.3 [time.cal.year.nonmembers]:

```
constexpr bool operator==(const year& x, const year& y) noexcept;
```

Returns: `int {x } == int {y }`.

~~`constexpr bool operator<(const year& x, const year& y) noexcept;`~~

~~*Returns:* `int {x } < int {y };`~~

```
constexpr strong_ordering operator<=>(const year& x, const year& y) noexcept;
```

Returns: `int {x } <=> int {y }`.

Change 27.8.9.3 [time.cal.md.nonmembers]:

```
constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
```

Returns: `x.month() == y.month() && x.day() == y.day()`.

~~`constexpr bool operator<(const month_day& x, const month_day& y) noexcept;`~~

~~*Returns:* If `x.month() < y.month()` returns `true`. Otherwise, if `x.month() > y.month()` returns `false`. Otherwise, returns `x.day() < y.day()`.~~

```
constexpr strong_ordering operator<=>(const month_day& x, const month_day& y) noexcept;
```

Effects: Equivalent to:

```
if (auto c = x.month() <=> y.month(); c != 0) return c;
return x.day() <=> y.day();
```

Change 27.8.10 [time.cal.mdlast]:

```
constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `x.month() == y.month()`.

~~`constexpr bool operator<(const month_day_last& x, const month_day_last& y) noexcept;`~~

~~*Returns:* `x.month() < y.month()`.~~

```
constexpr strong_ordering operator<=>(const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `x.month() <=> y.month()`.

Change 27.8.13.3 [time.cal.ym.nonmembers]:

```
constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month()`.

~~`constexpr bool operator<(const year_month& x, const year_month& y) noexcept;`~~

~~*Returns:* If `x.year() < y.year()` returns `true`. Otherwise, if `x.year() > y.year()` returns `false`. Otherwise, returns `x.month() < y.month()`.~~

```
constexpr strong_ordering operator<=>(const year_month& x, const year_month& y) noexcept;
```

Effects: Equivalent to:

```
if (auto c = x.year() <=> y.year(); c != 0) return c;
return x.month() <=> y.month();
```

Change 27.8.14.3 [time.cal.md.nonmembers]:

```
constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month() && x.day() == y.day()`.

~~constexpr bool operator<(const year_month_day& x, const year_month_day& y) noexcept;~~

2 ~~*Returns*: If x .year() < y.year(), returns true. Otherwise, if x.year() > y.year(), returns false. Otherwise, if x.month() < y.month() returns true. Otherwise, if x.month() > y.month() returns false. Otherwise, returns x.day() < y.day().~~

constexpr strong_ordering operator<=>(const year_month_day& x, const year_month_day& y) noexcept;

3 *Effects*: Equivalent to:

```
if (auto c = x.year() <=> y.year(); c != 0) return c;
if (auto c = x.month() <=> y.month(); c != 0) return c;
return x.day() <=> y.day();
```

Change 27.8.15.3 [time.cal.ymdlast.nonmembers]:

constexpr bool operator==(const year_month_day_last& x, const year_month_day_last& y) noexcept;

1 *Returns*: `x.year() == y.year() && x.month_day_last() == y.month_day_last()`.

~~constexpr bool operator<(const year_month_day_last& x, const year_month_day_last& y) noexcept;~~

2 ~~*Returns*: If x.year() < y.year(), returns true. Otherwise, if x.year() > y.year(), returns false. Otherwise, returns x.month_day_last() < y.month_day_last().~~

constexpr strong_ordering operator<=>(const year_month_day_last& x, const year_month_day_last& y) noexcept;

3 *Effects*: Equivalent to:

```
if (auto c = x.year() <=> y.year(); c != 0) return c;
return x.month_day_last() <=> y.month_day_last();
```

Change 27.10.5.3 [time.zone.nonmembers]:

constexpr bool operator==(const time_zone& x, const time_zone& y) noexcept;

1 *Returns*: `x.name() == y.name()`.

~~constexpr bool operator<(const time_zone& x, const time_zone& y) noexcept;~~

2 ~~*Returns*: x.name() < y.name().~~

constexpr strong_ordering operator<=>(const time_zone& x, const time_zone& y) noexcept;

3 *Returns*: `x.name() <=> y.name()`.

Change 27.10.7.4 [time.zone.zonedtime.nonmembers]:

```
template<class Duration1, class Duration2, class TimeZonePtr>
  bool operator==(const zoned_time<Duration1, TimeZonePtr>& x,
                  const zoned_time<Duration2, TimeZonePtr>& y);
```

1 *Returns*: `x.zone_ == y.zone_ && x.tp_ == y.tp_`.

~~template<class Duration1, class Duration2, class TimeZonePtr>~~
~~bool operator!=(const zoned_time<Duration1, TimeZonePtr>& x,~~
~~const zoned_time<Duration2, TimeZonePtr>& y);~~

2 ~~*Returns*: !(x == y).~~

Change 27.10.8.3 [time.zone.leap.nonmembers]:

constexpr bool operator==(const leap& x, const leap& y) noexcept;

1 *Returns*: `x.date() == y.date()`.

~~constexpr bool operator<(const leap& x, const leap& y) noexcept;~~

2 ~~*Returns*: x.date() < y.date().~~

constexpr strong_ordering operator<=>(const leap& x, const leap& y) noexcept;

2a *Returns*: `x.date() <=> y.date()`.

```
template<class Duration>
  constexpr bool operator==(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: x .date () == y.

```
template<class Duration>
  constexpr bool operator==(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: y == x.

```
template<class Duration>
  constexpr bool operator!=(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: !(x == y).

```
template<class Duration>
  constexpr bool operator!=(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: !(x == y).

```
template<class Duration>
  constexpr bool operator<(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: x .date () < y.

```
template<class Duration>
  constexpr bool operator<(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: x ≤ y .date ().

```
template<class Duration>
  constexpr bool operator>(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: y < x.

```
template<class Duration>
  constexpr bool operator>(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: y ≤ x.

```
template<class Duration>
  constexpr bool operator<=(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: !(y < x).

```
template<class Duration>
  constexpr bool operator<=(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: !(y < x).

```
template<class Duration>
  constexpr bool operator>=(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: !(x ≤ y).

```
template<class Duration>
  constexpr bool operator>=(const sys_time<Duration>& x, const leap& y) noexcept;
```

Returns: !(x ≤ y).

```
template<class Duration>
  constexpr auto operator<=>(const leap& x, const sys_time<Duration>& y) noexcept;
```

Returns: x .date () <=> y.

Change 27.10.9.3 [time.zone.link.nonmembers]:

```
constexpr bool operator==(const link& x, const link& y) noexcept;
```

Returns: x .name () == y .name ().

```
constexpr bool operator<(const link& x, const link& y) noexcept;
```

Returns: x .name () < y .name ().

```
constexpr strong_ordering operator<=>(const link& x, const link& y) noexcept;
```

Returns: x .name () <=> y .name ().

§ 5.13 Clause 28: Localization library

Remove the `!=` from `locale`.

Change 28.3.1 [locale]:

```
namespace std {
  class locale {
    [...]

    bool operator==(const locale& other) const;
-   bool operator!=(const locale& other) const;

    [...]
  };
}
```

Change 28.3.1.4 [locale.operators]:

```
bool operator==(const locale& other) const;
```

1 *Returns*: `true` if both arguments are the same locale, or one is a copy of the other, or each has a name and the names are identical; `false` otherwise.

```
~~bool operator!=(const locale& other) const;~~
```

2 ~~*Returns:* `!(*this == other)`.~~

§ 5.14 Clause 29: Input/output library

Add `==` to `space_info` and `file_status`, replace the relational operators with `<=>` for `path` and `directory_entry`.

Change 29.11.5 [fs.filesystem.syn]:

```
namespace std::filesystem {
  [...]

  struct space_info {
    uintmax_t capacity;
    uintmax_t free;
    uintmax_t available;

+   friend bool operator==(const space_info&, const space_info&) = default;
  };

  [...]
}
```

Change 29.11.7 [fs.class.path]:

```
namespace std::filesystem {
  class path {
    [...]

    // [fs.path.nonmember], non-member operators
    friend bool operator==(const path& lhs, const path& rhs) noexcept;
-   friend bool operator!=(const path& lhs, const path& rhs) noexcept;
-   friend bool operator< (const path& lhs, const path& rhs) noexcept;
-   friend bool operator<=(const path& lhs, const path& rhs) noexcept;
-   friend bool operator> (const path& lhs, const path& rhs) noexcept;
-   friend bool operator>=(const path& lhs, const path& rhs) noexcept;
+   friend strong_ordering operator<=>(const path& lhs, const path& rhs) noexcept;

    [...]
  };
}
```

Change 29.11.7.7 [fs.path.nonmember]:

```
friend bool operator==(const path& lhs, const path& rhs) noexcept;
```

3 *Returns*: ~~`!(lhs < rhs) && !(rhs < lhs)`~~ `lhs.compare(rhs) == 0`.

4 [*Note*: Path equality and path equivalence have different semantics.

(4.1) — Equality is determined by the path non-member `operator==`, which considers the two paths' lexical representations only. [*Example:* `path("foo") == "bar"` *is never* `true`. —end example*]

(4.2) — Equivalence is determined by the `equivalent()` non-member function, which determines if two paths resolve ([fs.class.path]) to the same file system entity. [*Example*: `equivalent("foo", "bar")` will be true when both paths resolve to the same file. —*end example*]

Programmers wishing to determine if two paths are “the same” must decide if “the same” means “the same representation” or “resolve to the same actual file”, and choose the appropriate function accordingly. —*end note*]

~~friend bool operator!=(const path& lhs, const path& rhs) noexcept;~~

~~*Returns:* !(lhs == rhs);~~

~~friend bool operator< (const path& lhs, const path& rhs) noexcept;~~

~~*Returns:* lhs.compare(rhs) < 0;~~

~~friend bool operator<=(const path& lhs, const path& rhs) noexcept;~~

~~*Returns:* !(rhs < lhs);~~

~~friend bool operator> (const path& lhs, const path& rhs) noexcept;~~

~~*Returns:* rhs < lhs;~~

~~friend bool operator>=(const path& lhs, const path& rhs) noexcept;~~

~~*Returns:* !(lhs < rhs);~~

```
friend strong_ordering operator<=>(const path& lhs, const path& rhs) noexcept;
```

Returns: lhs.compare(rhs) <=> 0.

Change 29.11.10 [fs.class.file.status]:

```
namespace std::filesystem {
  class file_status {
    [...]

+   friend bool operator==(const file_status& lhs, const file_status& rhs) noexcept
+     { return lhs.type() == rhs.type() && lhs.permissions() == rhs.permissions(); }
  };
}
```

Change 29.11.11 [fs.class.directory.entry]:

```
namespace std::filesystem {
  class directory_entry {
    [...]

    bool operator==(const directory_entry& rhs) const noexcept;
-   bool operator!=(const directory_entry& rhs) const noexcept;
-   bool operator< (const directory_entry& rhs) const noexcept;
-   bool operator> (const directory_entry& rhs) const noexcept;
-   bool operator<=(const directory_entry& rhs) const noexcept;
-   bool operator>=(const directory_entry& rhs) const noexcept;
+   strong_ordering operator<=>(const directory_entry& rhs) const noexcept;

    [...]
  };
}
```

Change 29.11.11.3 [fs.dir.entry.obs]:

```
bool operator==(const directory_entry& rhs) const noexcept;
```

Returns: pathobject == rhs.pathobject.

~~bool operator!=(const directory_entry& rhs) const noexcept;~~

~~*Returns:* pathobject != rhs.pathobject;~~

~~bool operator< (const directory_entry& rhs) const noexcept;~~

~~*Returns:* pathobject < rhs.pathobject;~~

~~bool operator> (const directory_entry& rhs) const noexcept;~~

~~*Returns:* pathobject > rhs.pathobject;~~

~~bool operator<=(const directory_entry& rhs) const noexcept;~~

~~*Returns:* pathobject <= rhs.pathobject;~~

~~bool operator>=(const directory_entry& rhs) const noexcept;~~

~~*Returns:* pathobject >= rhs.pathobject;~~

```
        strong_ordering operator<=>(const directory_entry& rhs) const noexcept;
```

37 *Returns*: `pathobject <=> rhs.pathobject`.

§ 5.15 Clause 30: Regular expressions library

Reduce all the `sub_match` operators to just `==` and `<=>`, and remove `!=` from `match_results`.

Change 30.4 [re.syn]:

```
        #include <initializer_list>

        namespace std {
          [...]

          // [re.submatch], class template sub_match
          template<class BidirectionalIterator>
            class sub_match;

          using csub_match  = sub_match<const char*>;
          using wcsub_match = sub_match<const wchar_t*>;
          using ssub_match  = sub_match<string::const_iterator>;
          using wssub_match = sub_match<wstring::const_iterator>;

          // [re.submatch.op], sub_match non-member operators
          template<class BiIter>
            bool operator==(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
        - template<class BiIter>
        -   bool operator!=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
        - template<class BiIter>
        -   bool operator<(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
        - template<class BiIter>
        -   bool operator>(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
        - template<class BiIter>
        -   bool operator<=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
        - template<class BiIter>
        -   bool operator>=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
        + template<class BiIter>
        +   constexpr auto operator<=>(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);

        - template<class BiIter, class ST, class SA>
        -   bool operator==(
        -     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        -     const sub_match<BiIter>& rhs);
        - template<class BiIter, class ST, class SA>
        -   bool operator!=(
        -     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        -     const sub_match<BiIter>& rhs);
        - template<class BiIter, class ST, class SA>
        -   bool operator<(
        -     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        -     const sub_match<BiIter>& rhs);
        - template<class BiIter, class ST, class SA>
        -   bool operator>(
        -     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        -     const sub_match<BiIter>& rhs);
        - template<class BiIter, class ST, class SA>
        -   bool operator<=(
        -     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        -     const sub_match<BiIter>& rhs);
        - template<class BiIter, class ST, class SA>
        -   bool operator>=(
        -     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
        -     const sub_match<BiIter>& rhs);

          template<class BiIter, class ST, class SA>
            bool operator==(
              const sub_match<BiIter>& lhs,
              const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
        - template<class BiIter, class ST, class SA>
        -   bool operator!=(
        -     const sub_match<BiIter>& lhs,
        -     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
        - template<class BiIter, class ST, class SA>
        -   bool operator<(
        -     const sub_match<BiIter>& lhs,
        -     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
        - template<class BiIter, class ST, class SA>
        -   bool operator>(
        -     const sub_match<BiIter>& lhs,
        -     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
        - template<class BiIter, class ST, class SA>
```

```
-   bool operator<=(
-     const sub_match<BiIter>& lhs,
-     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
- template<class BiIter, class ST, class SA>
-   bool operator>=(
-     const sub_match<BiIter>& lhs,
-     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
+ template<class BiIter, class ST, class SA>
+   auto operator<=>(
+     const sub_match<BiIter>& lhs,
+     const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);

- template<class BiIter>
-   bool operator==(const typename iterator_traits<BiIter>::value_type* lhs,
-                   const sub_match<BiIter>& rhs);
- template<class BiIter>
-   bool operator!=(const typename iterator_traits<BiIter>::value_type* lhs,
-                   const sub_match<BiIter>& rhs);
- template<class BiIter>
-   bool operator<(const typename iterator_traits<BiIter>::value_type* lhs,
-                  const sub_match<BiIter>& rhs);
- template<class BiIter>
-   bool operator>(const typename iterator_traits<BiIter>::value_type* lhs,
-                  const sub_match<BiIter>& rhs);
- template<class BiIter>
-   bool operator<=(const typename iterator_traits<BiIter>::value_type* lhs,
-                   const sub_match<BiIter>& rhs);
- template<class BiIter>
-   bool operator>=(const typename iterator_traits<BiIter>::value_type* lhs,
-                   const sub_match<BiIter>& rhs);

  template<class BiIter>
    bool operator==(const sub_match<BiIter>& lhs,
                    const typename iterator_traits<BiIter>::value_type* rhs);
- template<class BiIter>
-   bool operator!=(const sub_match<BiIter>& lhs,
-                   const typename iterator_traits<BiIter>::value_type* rhs);
- template<class BiIter>
-   bool operator<(const sub_match<BiIter>& lhs,
-                  const typename iterator_traits<BiIter>::value_type* rhs);
- template<class BiIter>
-   bool operator>(const sub_match<BiIter>& lhs,
-                  const typename iterator_traits<BiIter>::value_type* rhs);
- template<class BiIter>
-   bool operator<=(const sub_match<BiIter>& lhs,
-                   const typename iterator_traits<BiIter>::value_type* rhs);
- template<class BiIter>
-   bool operator>=(const sub_match<BiIter>& lhs,
-                   const typename iterator_traits<BiIter>::value_type* rhs);
+ template<class BiIter>
+   auto operator<=>(const sub_match<BiIter>& lhs,
+                    const typename iterator_traits<BiIter>::value_type* rhs);

- template<class BiIter>
-   bool operator==(const typename iterator_traits<BiIter>::value_type& lhs,
-                   const sub_match<BiIter>& rhs);
- template<class BiIter>
-   bool operator!=(const typename iterator_traits<BiIter>::value_type& lhs,
-                   const sub_match<BiIter>& rhs);
- template<class BiIter>
-   bool operator<(const typename iterator_traits<BiIter>::value_type& lhs,
-                  const sub_match<BiIter>& rhs);
- template<class BiIter>
-   bool operator>(const typename iterator_traits<BiIter>::value_type& lhs,
-                  const sub_match<BiIter>& rhs);
- template<class BiIter>
-   bool operator<=(const typename iterator_traits<BiIter>::value_type& lhs,
-                   const sub_match<BiIter>& rhs);
- template<class BiIter>
-   bool operator>=(const typename iterator_traits<BiIter>::value_type& lhs,
-                   const sub_match<BiIter>& rhs);

  template<class BiIter>
    bool operator==(const sub_match<BiIter>& lhs,
                    const typename iterator_traits<BiIter>::value_type& rhs);
- template<class BiIter>
-   bool operator!=(const sub_match<BiIter>& lhs,
-                   const typename iterator_traits<BiIter>::value_type& rhs);
- template<class BiIter>
-   bool operator<(const sub_match<BiIter>& lhs,
-                  const typename iterator_traits<BiIter>::value_type& rhs);
- template<class BiIter>
-   bool operator>(const sub_match<BiIter>& lhs,
```

```
-                   const typename iterator_traits<BiIter>::value_type& rhs);
- template<class BiIter>
-   bool operator<=(const sub_match<BiIter>& lhs,
-                   const typename iterator_traits<BiIter>::value_type& rhs);
- template<class BiIter>
-   bool operator>=(const sub_match<BiIter>& lhs,
-                   const typename iterator_traits<BiIter>::value_type& rhs);
+ template<class BiIter>
+   auto operator<=>(const sub_match<BiIter>& lhs,
+                    const typename iterator_traits<BiIter>::value_type& rhs);

  template<class charT, class ST, class BiIter>
    basic_ostream<charT, ST>&
      operator<<(basic_ostream<charT, ST>& os, const sub_match<BiIter>& m);

  // [re.results], class template match_results
  template<class BidirectionalIterator,
           class Allocator = allocator<sub_match<BidirectionalIterator>>>
    class match_results;

  using cmatch  = match_results<const char*>;
  using wcmatch = match_results<const wchar_t*>;
  using smatch  = match_results<string::const_iterator>;
  using wsmatch = match_results<wstring::const_iterator>;

  // match_results comparisons
  template<class BidirectionalIterator, class Allocator>
    bool operator==(const match_results<BidirectionalIterator, Allocator>& m1,
                    const match_results<BidirectionalIterator, Allocator>& m2);
- template<class BidirectionalIterator, class Allocator>
-   bool operator!=(const match_results<BidirectionalIterator, Allocator>& m1,
-                   const match_results<BidirectionalIterator, Allocator>& m2);

  [...]
}
```

Change 30.9.2 [re.submatch.op]. As a result, there should be nine functions left here: four `operator==`s, four `operator<=>`s, and the `operator<<`.

0 Let *SM_CAT*(I) be `compare_three_way_result_t<basic_string<typename iterator_traits<I>::value_type>>`.

```
template<class BiIter>
  bool operator==(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

1 *Returns*: `lhs.compare(rhs) == 0`.

```
template<class BiIter>
  bool operator!=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

2 ~~*Returns:* `lhs.compare(rhs) != 0`.~~

```
template<class BiIter>
  bool operator<(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

3 ~~*Returns:* `lhs.compare(rhs) < 0`.~~

```
template<class BiIter>
  bool operator>(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

4 ~~*Returns:* `lhs.compare(rhs) >= 0`.~~

```
template<class BiIter>
  bool operator<=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

5 ~~*Returns:* `lhs.compare(rhs) <= 0`.~~

```
template<class BiIter>
  bool operator>=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

6 ~~*Returns:* `lhs.compare(rhs) >= 0`.~~

```
template<class BiIter>
  auto operator<=>(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

a *Returns*: `static_cast<SM_CAT(BiIter)>(lhs.compare(rhs) >= 0)`.

```
template<class BiIter, class ST, class SA>
  bool operator==(
    const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
```

Returns:

```
rhs.compare(typename sub_match<BiIter>::string_type(lhs.data(), lhs.size())) == 0
```

```
template<class BiIter, class ST, class SA>
  bool operator!=(
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
      const sub_match<BiIter>& rhs);
```

Returns: `!(lhs == rhs)`.

```
template<class BiIter, class ST, class SA>
  bool operator<(
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
      const sub_match<BiIter>& rhs);
```

Returns:

```
rhs.compare(typename sub_match<BiIter>::string_type(lhs.data(), lhs.size())) > 0
```

```
template<class BiIter, class ST, class SA>
  bool operator>(
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
      const sub_match<BiIter>& rhs);
```

Returns: `rhs < lhs`.

```
template<class BiIter, class ST, class SA>
  bool operator<=(
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
      const sub_match<BiIter>& rhs);
```

Returns: `!(rhs < lhs)`.

```
template<class BiIter, class ST, class SA>
  bool operator>=(
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
      const sub_match<BiIter>& rhs);
```

Returns: `!(lhs < rhs)`.

```
template<class BiIter, class ST, class SA>
  bool operator==(
      const sub_match<BiIter>& lhs,
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

Returns:

```
lhs.compare(typename sub_match<BiIter>::string_type(rhs.data(), rhs.size())) == 0
```

```
template<class BiIter, class ST, class SA>
  bool operator!=(
      const sub_match<BiIter>& lhs,
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

Returns: `!(lhs == rhs)`.

```
template<class BiIter, class ST, class SA>
  bool operator<(
      const sub_match<BiIter>& lhs,
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

Returns:

```
lhs.compare(typename sub_match<BiIter>::string_type(rhs.data(), rhs.size())) < 0
```

```
template<class BiIter, class ST, class SA>
  bool operator>(
      const sub_match<BiIter>& lhs,
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

Returns: `rhs < lhs`.

```
template<class BiIter, class ST, class SA>
  bool operator<=(
      const sub_match<BiIter>& lhs,
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

Returns: `!(rhs < lhs)`.

```
template<class BiIter, class ST, class SA>
  bool operator>=(
```

```
      const sub_match<BiIter>& lhs,
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

18 *Returns:* `!(lhs <= rhs);`

```
template<class BiIter, class ST, class SA>
  auto operator<=>(
      const sub_match<BiIter>& lhs,
      const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

b *Returns*:

```
 static_cast<SM_CAT(BiIter)>(lhs.compare(
     typename sub_match<BiIter>::string_type(rhs.data(), rhs.size()))
       <=> 0
     )
```

```
template<class BiIter>
  bool operator==(const typename iterator_traits<BiIter>::value_type* lhs,
                  const sub_match<BiIter>& rhs);
```

19 *Returns:* `lhs.compare(lhs) == 0;`

```
template<class BiIter>
  bool operator!=(const typename iterator_traits<BiIter>::value_type* lhs,
                  const sub_match<BiIter>& rhs);
```

20 *Returns:* `!(lhs == rhs);`

```
template<class BiIter>
  bool operator<(const typename iterator_traits<BiIter>::value_type* lhs,
                 const sub_match<BiIter>& rhs);
```

21 *Returns:* `rhs.compare(lhs) >= 0;`

```
template<class BiIter>
  bool operator>(const typename iterator_traits<BiIter>::value_type* lhs,
                 const sub_match<BiIter>& rhs);
```

22 *Returns:* `rhs < lhs;`

```
template<class BiIter>
  bool operator<=(const typename iterator_traits<BiIter>::value_type* lhs,
                  const sub_match<BiIter>& rhs);
```

23 *Returns:* `!(rhs < lhs);`

```
template<class BiIter>
  bool operator>=(const typename iterator_traits<BiIter>::value_type* lhs,
                  const sub_match<BiIter>& rhs);
```

24 *Returns:* `!(lhs < rhs);`

```
template<class BiIter>
  bool operator==(const sub_match<BiIter>& lhs,
                  const typename iterator_traits<BiIter>::value_type* rhs);
```

25 *Returns*: `lhs.compare(rhs) == 0`.

```
template<class BiIter>
  bool operator!=(const sub_match<BiIter>& lhs,
                  const typename iterator_traits<BiIter>::value_type* rhs);
```

26 *Returns:* `!(lhs == rhs);`

```
template<class BiIter>
  bool operator<(const sub_match<BiIter>& lhs,
                 const typename iterator_traits<BiIter>::value_type* rhs);
```

27 *Returns:* `lhs.compare(rhs) <= 0;`

```
template<class BiIter>
  bool operator>(const sub_match<BiIter>& lhs,
                 const typename iterator_traits<BiIter>::value_type* rhs);
```

28 *Returns:* `rhs < lhs;`

```
template<class BiIter>
  bool operator<=(const sub_match<BiIter>& lhs,
                  const typename iterator_traits<BiIter>::value_type* rhs);
```

29 *Returns:* `!(rhs < lhs);`

```
template<class BiIter>
  bool operator>=(const sub_match<BiIter>& lhs,
                  const typename iterator_traits<BiIter>::value_type* rhs);
```

Returns: !(lhs < rhs).

```
template<class BiIter>
  auto operator<=>(const sub_match<BiIter>& lhs,
                   const typename iterator_traits<BiIter>::value_type* rhs);
```

Returns: static_cast<SM_CAT(BiIter)>(lhs.compare(rhs) <=> 0).

```
template<class BiIter>
  bool operator==(const typename iterator_traits<BiIter>::value_type& lhs,
                  const sub_match<BiIter>& rhs);
```

Returns: rhs == lhs.

```
template<class BiIter>
  bool operator!=(const typename iterator_traits<BiIter>::value_type& lhs,
                  const sub_match<BiIter>& rhs);
```

Returns: !(lhs == rhs).

```
template<class BiIter>
  bool operator<(const typename iterator_traits<BiIter>::value_type& lhs,
                 const sub_match<BiIter>& rhs);
```

Returns: rhs.compare(typename sub_match<BiIter>::string_type(1, lhs)) == 0.

```
template<class BiIter>
  bool operator>(const typename iterator_traits<BiIter>::value_type& lhs,
                 const sub_match<BiIter>& rhs);
```

Returns: rhs < lhs.

```
template<class BiIter>
  bool operator<=(const typename iterator_traits<BiIter>::value_type& lhs,
                  const sub_match<BiIter>& rhs);
```

Returns: !(rhs < lhs).

```
template<class BiIter>
  bool operator>=(const typename iterator_traits<BiIter>::value_type& lhs,
                  const sub_match<BiIter>& rhs);
```

Returns: !(lhs < rhs).

```
template<class BiIter>
  bool operator==(const sub_match<BiIter>& lhs,
                  const typename iterator_traits<BiIter>::value_type& rhs);
```

Returns: lhs.compare(typename sub_match<BiIter>::string_type(1, rhs)) == 0.

```
template<class BiIter>
  bool operator!=(const sub_match<BiIter>& lhs,
                  const typename iterator_traits<BiIter>::value_type& rhs);
```

Returns: !(lhs == rhs).

```
template<class BiIter>
  bool operator<(const sub_match<BiIter>& lhs,
                 const typename iterator_traits<BiIter>::value_type& rhs);
```

Returns: lhs.compare(typename sub_match<BiIter>::string_type(1, rhs)) < 0.

```
template<class BiIter>
  bool operator>(const sub_match<BiIter>& lhs,
                 const typename iterator_traits<BiIter>::value_type& rhs);
```

Returns: rhs < lhs.

```
template<class BiIter>
  bool operator<=(const sub_match<BiIter>& lhs,
                  const typename iterator_traits<BiIter>::value_type& rhs);
```

~~*Returns:* `!(rhs < lhs);`~~

```
~~template<class BiIter>~~
~~  bool operator>=(const sub_match<BiIter>& lhs,~~
~~                  const typename iterator_traits<BiIter>::value_type& rhs);~~
```

~~*Returns:* `!(lhs < rhs);`~~

```
template<class BiIter>
  auto operator<=>(const sub_match<BiIter>& lhs,
                   const typename iterator_traits<BiIter>::value_type& rhs);
```

Returns:

```
        static_cast<SM_CAT(BiIter)>(lhs.compare(
            typename sub_match<BiIter>::string_type(1, rhs))
              <=> 0
            )
```

```
template<class charT, class ST, class BiIter>
  basic_ostream<charT, ST>&
    operator<<(basic_ostream<charT, ST>& os, const sub_match<BiIter>& m);
```

Returns: `os << m .str ()`.

Remove the `!=` from 30.10.8 [re.results.nonmember]:

```
~~template<class BidirectionalIterator, class Allocator>~~
~~bool operator!=(const match_results<BidirectionalIterator, Allocator>& m1,~~
~~                const match_results<BidirectionalIterator, Allocator>& m2);~~
```

~~*Returns:* `!(m1 == m2);`~~

Change 30.12.1 [re.regiter]:

```
namespace std {
  template<class BidirectionalIterator,
           class charT = typename iterator_traits<BidirectionalIterator>::value_type,
           class traits = regex_traits<charT>>
    class regex_iterator {
      [...]
      regex_iterator& operator=(const regex_iterator&);
      bool operator==(const regex_iterator&) const;
-     bool operator!=(const regex_iterator&) const;
      const value_type& operator*() const;
      [...]
    };
}
```

Remove the `!=` from 30.12.1.2 [re.regiter.comp]:

```
~~bool operator!=(const regex_iterator& right) const;~~
```

~~*Returns:* `!(*this == right);`~~

Change 30.12.2 [re.tokiter]:

```
namespace std {
  template<class BidirectionalIterator,
           class charT = typename iterator_traits<BidirectionalIterator>::value_type,
           class traits = regex_traits<charT>>
    class regex_token_iterator {
      [...]
      bool operator==(const regex_token_iterator&) const;
-     bool operator!=(const regex_token_iterator&) const;
      [...]
    };
}
```

Remove the `!=` from 30.12.2.2 [re.tokiter.comp]:

```
~~bool operator!=(const regex_token_iterator& right) const;~~
```

~~*Returns:* `!(*this == right);`~~

5.16 Clause 31: Atomic operations library

No changes necessary.

5.17 Clause 32: Thread support library

Replace `thread::id`s operators with just `==` and `<=>`.

Change 32.3.2.1 [thread.thread.id]:

```
namespace std {
  class thread::id {
  public:
    id() noexcept;
  };

  bool operator==(thread::id x, thread::id y) noexcept;
- bool operator!=(thread::id x, thread::id y) noexcept;
- bool operator<(thread::id x, thread::id y) noexcept;
- bool operator>(thread::id x, thread::id y) noexcept;
- bool operator<=(thread::id x, thread::id y) noexcept;
- bool operator>=(thread::id x, thread::id y) noexcept;
+ strong_ordering operator<=>(thread::id x, thread::id y) noexcept;

  template<class charT, class traits>
    basic_ostream<charT, traits>&
      operator<<(basic_ostream<charT, traits>& out, thread::id id);

  // hash support
  template<class T> struct hash;
  template<> struct hash<thread::id>;
}
```

```
bool operator==(thread::id x, thread::id y) noexcept;
```

6 *Returns*: `true` only if `x` and `y` represent the same thread of execution or neither `x` nor `y` represents a thread of execution.

~~`bool operator!=(thread::id x, thread::id y) noexcept;`~~

7 ~~*Returns:* `!(x == y)`.~~

~~`bool operator<(thread::id x, thread::id y) noexcept;`~~

8 ~~*Returns:* A value such that `operator<` is a total ordering as described in [alg.sorting].~~

~~`bool operator>(thread::id x, thread::id y) noexcept;`~~

9 ~~*Returns:* `y < x`.~~

~~`bool operator<=(thread::id x, thread::id y) noexcept;`~~

10 ~~*Returns:* `!(y < x)`.~~

~~`bool operator>=(thread::id x, thread::id y) noexcept;`~~

11 ~~*Returns:* `!(x < y)`.~~

```
strong_ordering operator<=>(thread::id x, thread::id y) noexcept;
```

a *Returns*: A value such that `operator<=>` is a total ordering as described in [alg.sorting].

§ 6 References

[P0732R2] Jeff Snyder, Louis Dionne. 2018. Class Types in Non-Type Template Parameters.
https://wg21.link/p0732r2

[P0790R2] David Stone. 2019. Effect of operator<=> on the C++ Standard Library.
https://wg21.link/p0790r2

[P0891R2] Gašper Ažman, Jeff Snyder. 2019. Make `strong_order` a Customization Point!
https://wg21.link/p0891r2

[P1154R1] Arthur O'Dwyer, Jeff Snyder. 2019. Type traits for structural comparison.
https://wg21.link/p1154r1

[P1185R2] Barry Revzin. 2019. `<=>` `!=` `==`.
https://wg21.link/p1185r2

[P1186R1] Barry Revzin. 2019. When do you actually use `<=>`?
https://wg21.link/p1186r1

[P1186R2] Barry Revzin. 2019. When do you actually use `<=>`?
https://wg21.link/p1186r2

[P1188R0] Barry Revzin. 2019. Library utilities for `<=>`.
https://wg21.link/p1188r0

[P1189R0] Barry Revzin. 2019. Adding `<=>` to library.
https://wg21.link/p1189r0

[P1191R0] David Stone. 2018. Adding `operator <=>` to types that are not currently comparable.
https://wg21.link/p1191r0

[P1201R0] Oleg Fatkhiev, Antony Polukhin. 2018. Variant direct comparisons.
https://wg21.link/p1201r0

[P1295R0] Tomasz Kamiński. 2018. Spaceship library update.
https://wg21.link/p1295r0

[P1380R1] Lawrence Crowl. 2019. Ambiguity and Insecurities with Three-Way Comparison.
https://wg21.link/p1380r1

[P1614R0] Barry Revzin. 2019. The Mothership Has Landed: Adding `<=>` to the Library.
https://wg21.link/p1614r0